DEPARTMENT OF COMPUTER SCIENCE AND IT
NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

COMPLEX COMPUTING PROBLEM (CCP) REPORT

Course Code: CT-351
Course Title: Knowledge Representation & Reasoning

Project Title:	Smart Parking Access & Space Allocation Reasoning System
Selected Domain:	Smart parking: campus parking access and space allocation
Section:	A
Submission Date:	8 November 2026

GROUP MEMBERS

S. No.	Student Name	Roll Number	Assigned Contribution
1	Syed Mohmmad Huzaifa	AI-23022	Problem, knowledge acquisition, frames and script
2	Maaz ur Rehman	AI-23036	Semantic network, description logic, trade-offs
3	Owais Tariq	AI-23037	FOL, Horn knowledge base, forward chaining
4	Shaikh Muhammad Zain	AI-23306	Backward chaining, interfaces, testing, report

Instructor: Miss Dure Shahwar

CLO: CLO-3 (Design/Analysis) **PLO:** PLO-10 (Lifelong Learning)

Group: 12 **Semester:** 7th

Department of Computer Science and IT
Course Code: CT-351 | Course Title: Knowledge Representation & Reasoning
Complex Computing Problem (CCP): Grading Rubric

Student No.	Name	Roll No.
S1	Syed Mohmmad Huzaifa	AI-23022
S2	Maaz ur Rehman	AI-23036
S3	Owais Tariq	AI-23037
S4	Shaikh Muhammad Zain	AI-23306

Criteria and scales	0 (Unsatisfactory)	1 (Developing)	2 (Proficient)	S1	S2	S3	S4
Criterion 1: Problem Statement & Motivation	Objective of the project and domain requirements are not clear or described.	Some lack of clarity in objectives/purpose or domain justification.	The project's objectives and domain motivation are clearly stated and justified.				
Criterion 2: Structural KR (Frames & Semantic Networks)	Model fails to represent necessary concepts, frames, slots, or semantic relationships.	Model includes frames and semantic networks but lacks complete slots, facets, or default values.	Model meets Proficient criteria with comprehensive, accurate frame structures and semantic networks.				
Criterion 3: Formal Logic (FOL & Description Logic)	Logical representations contain major syntax/semantic errors or are missing.	FOL and DL formulas are present but contain minor syntax/quantifier errors.	Model represents domain constraints perfectly using correct FOL formulas and Description Logic axioms.				
Criterion 4: Inference Engine (Horn Clauses & Chaining)	Rules are not converted to Horn clauses; failure to execute chaining.	Rules are in Horn clauses, but Forward/Backward chaining traces have minor logical gaps.	Horn clauses are correctly formulated with flawless, detailed Forward and Backward chaining traces.				
Criterion 5: Team Participation & Individual Viva	The individual did not contribute to the project and failed to answer viva questions.	The individual contributed partially and demonstrated basic understanding during viva.	The individual contributed significantly and demonstrated mastery of KR concepts during viva.				
Total Marks (Out of 10):							

Teacher's Signature: _____ Date: _____

REPORT CONTENTS

1. INTRODUCTION 4
1.1 Selected Domain and Background 4
1.2 Problem Statement and Objectives 4
2. DOMAIN ANALYSIS 4
2.1 Domain Description and Requirements 4
2.2 Entities, Relationships, and Challenges 6
3. KNOWLEDGE REPRESENTATION 7
3.1 Frames 7
3.2 Semantic Networks 11
4. FORMAL LOGIC REPRESENTATION 12
4.1 First-Order Logic (FOL) 12
4.2 Description Logic (DL) 14
5. HORN CLAUSE KNOWLEDGE BASE 16
6. INFERENCE AND REASONING 19
6.1 Forward Chaining 20
6.2 Backward Chaining 22
7. IMPLEMENTATION 24
7.1 Tools and Technologies 24
7.2 Code / Execution 24
8. RESULTS AND DISCUSSION 28
8.1 Results 28
8.2 Discussion 30
9. CONCLUSION 31
9.1 Conclusion and Future Improvements 31
10. REFERENCES 31
11. TEAM PARTICIPATION 33

1. INTRODUCTION

1.1 Selected Domain and Background

The selected domain is smart parking: who may enter a university car park and which bay they get. Smart parking systems usually mean sensors and apps that show free bays. The harder part is deciding who is allowed to use which bay. That decision rests on policy (permits, zones, accessibility, reservations) and not on sensor data, and policy is exactly what knowledge representation is for: facts about the world, rules that say what follows from them, and a reasoner that applies the rules the same way every time and can show its working.

Three problems with the current manual process drove the project. Decisions are inconsistent, because two guards can apply the same rule differently. Decisions are unexplained, so a refused driver cannot tell whether the problem is their permit, the zone or the bay. Policy changes are slow, because they live in people's heads or in scattered code.

A database alone does not fix this (section 2.2.4). The domain also needs every paradigm in the course: a class hierarchy with defaults, a fixed sequence of events, typed relations, quantified constraints, disjoint classes and many if-then rules that must run fast.

1.2 Problem Statement and Objectives

Given structured facts about people, vehicles, permits, zones, bays and reservations, plus today's date, the current hour and a list of requests, decide for every request whether the driver may enter and which bay they get. Every "yes" must come with the chain of facts and rules that supports it, and every "no" must come with the first condition that failed. The policy must be written once, in a form that can be read, checked and changed without rewriting code.

The objectives are:

1. Represent the domain with frames (at least ten class frames with slots, facets and defaults), a script and a semantic network generated from the frames.
2. Write every rule in FOL with explicit quantifiers, plus at least ten constraints that go beyond Horn form, and check them on the data.
3. Write a DL TBox with subsumption, existential and universal restrictions, definitions and disjointness, and a reasoner that classifies it and finds contradictions.
4. Convert the policy into Horn clauses and run it with working forward and backward chaining engines that produce step-by-step traces.
5. Pass at least eight test cases covering the domain requirements with both engines, and show that the two engines derive the same facts.

The system covers one campus car park with three zones and nine bays, eight people and one day of requests. It decides access and allocation and explains each decision. It does not read live sensors, take payments, plan routes, handle hourly reservations or learn from data. Times are whole hours and dates are YYYYMMDD integers.

The solution has four layers (Figure 1). The representation layer holds the frames, script, semantic network, FOL statements and DL TBox. All of them feed one knowledge base of facts and Horn rules. The inference layer has the forward and backward chainers, the why-not explainer, the DL reasoner and the FOL model checker. On top sit a command-line interface, a Streamlit web interface and a public website (section 7.2.9). A build script regenerates the Prolog program, the OWL ontology, the trace sheets and this report from the code, so the report cannot drift from what the engines do.

2. DOMAIN ANALYSIS

2.1 Domain Description and Requirements

2.1.1 Smart parking domain

The test campus is modelled on the NED main campus car park. It has one facility (ned_lot) with three zones: zone_a for students (open 07:00-22:00, five bays), zone_e for employees (06:00-23:00, three bays) and zone_v for visitors (08:00-17:00, one bay). The bays include an accessible bay, a motorbike bay, an EV charging bay, one occupied bay and two reserved bays. Eight people use it. Ali, Sara, Usman

and Zara are students; Sara holds an accessibility badge and Usman's permit expired on 15 September 2026. Dr. Ahmed is faculty and drives an electric car, Fatima is staff and rides a motorbike, and Bilal and Hina are visitors, only Bilal with a pass. The main scenario is the morning rush of 2 October 2026, with eight requests between 08:00 and 09:20.

2.1.2 Knowledge acquisition

We collected the kind of documents a university parking office publishes: permit categories and validity, zone signs and hours, the accessible parking policy, the reservation procedure and the rule for disputed bays. Where NED does not publish a rule, we wrote down an explicit project assumption (for example the three-level priority scale).

Two of us then played the parking officer and the gate guard while the others asked "what happens if" questions. Every answer was written as an if-then sentence. "Can a student park with an expired permit?" became "a permit is valid if its status is active and it has not expired", which is R06. Nouns in these sentences became classes, categories became slot values, verbs became relations, and repeated patterns ("faculty and staff both...") became superclasses such as Employee.

Finally every requirement became a test scenario with an expected answer, and we fixed the knowledge base until all of them passed. One error found this way: the first draft let an ineligible driver make others lose a conflict, so R25 and R26 now require the winner to satisfy can_allocate.

2.1.3 Domain requirements

The rules below came out of knowledge acquisition. The rule numbers in brackets are where each requirement is implemented.

1. General parking needs a valid permit: status active, and today on or after its start date and on or before its expiry date (R06, R10).
2. A permit counts only for the kind of person it was issued to: student permits for students, employee permits for faculty and staff, visitor passes for visitors (R07-R09).
3. A visitor without a one-day visitor pass is not authorized (R09).
4. Students may use only student zones, employees only the employee zone, visitors only the visitor zone (R13 with the grants/2 table).
5. Nobody may enter a zone outside its opening hours (R12).
6. A vehicle gets only a bay it fits: cars use standard bays, electric cars also use EV bays, motorbikes use motorbike bays (R14 with the fits/2 table).
7. Accessible bays are only for accessibility badge holders (R16).
8. An occupied bay is never allocated (R17, R19, IC3).
9. A reserved bay goes only to the person holding today's reservation for it (R18, R20).
10. A person with two reservations on the same day gets nothing until the conflict is resolved (R27, R28).
11. When several eligible drivers want one bay, the highest priority wins (badge holders 3, employees 2, students and visitors 1), and the earlier request breaks a tie (R21-R26).
12. A bay never goes to two people and nobody gets two bays (IC1, IC2).
13. Nobody is both a student and an employee, a student and a visitor, an employee and a visitor, or faculty and staff (IC4-IC7, DL axioms D01-D04).

2.1.4 Domain constraints

The hard constraints are requirements 8, 12 and 13: no occupied bay is allocated, no bay goes to two people, nobody gets two bays and the person classes are disjoint. These are checked after every run as denial clauses IC1-IC7. The policy also has exceptions that the rules must handle: a reserved bay is not free but may still go to its holder (R20), an employee permit opens a zone a student permit does not (R13), and an accessible bay is suitable for a car but still needs a badge (R16).

2.2 Entities, Relationships, and Challenges

2.2.1 Domain entities

Table 1: Domain entities

Entity	Description	Important attributes (frame slots)
Person	Anyone who may drive onto campus	fullName, accessibilityBadge, owns, hasPermit
Student	Enrolled student (Ali, Sara, Usman, Zara)	studentId, department
Faculty / Staff	Employees: Dr. Ahmed (faculty), Fatima (staff)	employeeId, department, designation / jobTitle
Visitor	Guest without a university permit (Bilal, Hina)	purpose, hostDepartment
Vehicle	Registered car, electric car or motorbike	plateNo, vehicleType
Parking Permit	Student permit, employee permit or one-day visitor pass	permitClass, status, startDate, expiryDate
Parking Space	One bay; standard, accessible, EV charging or motorbike	locatedIn, spaceType, status, zoneType
Parking Zone	Group of bays for one kind of user, with opening hours	zoneType, openHour, closeHour, partOf, capacity
Reservation	A person's booking of one bay for one day	madeBy, forSpace, date

2.2.2 Domain relationships

People are related to the class hierarchy by is-a (Student is-a Person, Faculty is-a Employee). A person owns vehicles and has-a permit. A bay is located in (part of) a zone, which is part of the facility. A reservation is made by a person for a bay on a date, so it is a reified three-way relation. Value relations tie the policy together: a permit class grants a zone type and a vehicle type fits a space type. The reasoner derives further relations from these, the most important being eligible-for (eligible_for_space) between a person and a bay, and allocate between a driver and the bay they get.

Person has three subclasses, Student, Employee and Visitor, and Employee has two, Faculty and Staff. ParkingSpace has AccessibleSpace, EVChargingSpace and MotorbikeSpace. Vehicle, ParkingPermit, ParkingZone, ParkingFacility and Reservation have no subclasses as frames; in the DL layer, Vehicle splits into Car, ElectricCar and Motorbike and ParkingPermit into StudentPermit, EmployeePermit, VisitorPass and ValidPermit. Value concepts (VehicleType, SpaceType, ZoneType, PermitClass) hold the allowed symbolic values. The frame hierarchy is drawn in section 3.1.1.

2.2.3 Challenges in representation and reasoning

- Defaults and overriding. Most people have no badge and most vehicles are cars, and a subclass can change a default. Logic cannot override a conclusion, so defaults have to be settled before the rules run (section 3.1.1).
- Exceptions. A reserved bay is not free, yet its holder may still get it. An accessible bay suits a car, yet the driver still needs a badge.
- "Unless" in the policy. A driver gets a bay unless someone with higher priority wants it. A definite clause cannot contain negation, so this needs negation as failure with stratification (section 5.4).
- Competing requests. The answer for one driver depends on who else asked for the same bay, so priority and arrival time have to be compared across requests.
- Missing information. The gate must treat an unrecorded permit as "no", while a shared ontology must treat it as "not known" (section 4.2.7).
- Expressive power against speed. Some constraints need ∃ or disjunction in the conclusion, but the gate needs an answer that is guaranteed to arrive (section 8.2.2).
- Explanation. A bare "no" does not help a driver, so every refusal has to name the first condition that failed.

2.2.4 Limitations of traditional database systems

A relational database could store every fact here and is better than our system at storage, concurrency and large queries. It falls short on reasoning:

- It answers only what is stored. Concluding that Dr. Ahmed is a person because faculty are employees and employees are people needs a join written by hand into every query; the reasoner derives it once from R04 and R02.
- Business rules end up spread over application code, triggers and stored procedures, where they cannot be listed or checked together. Here all 28 rules are in one file and printed in section 5.3.

- A query result carries no explanation. The backward chainer returns a proof tree for every "yes" and the failing condition for every "no".
- Check constraints cannot express statements such as "whoever gets an accessible bay holds a badge" (F07), which quantify across several tables.
- A database works under the closed-world assumption only. The DL layer also supports the open-world reading (section 4.2.7), which can tell "not authorized" from "not known to be authorized".

3. KNOWLEDGE REPRESENTATION

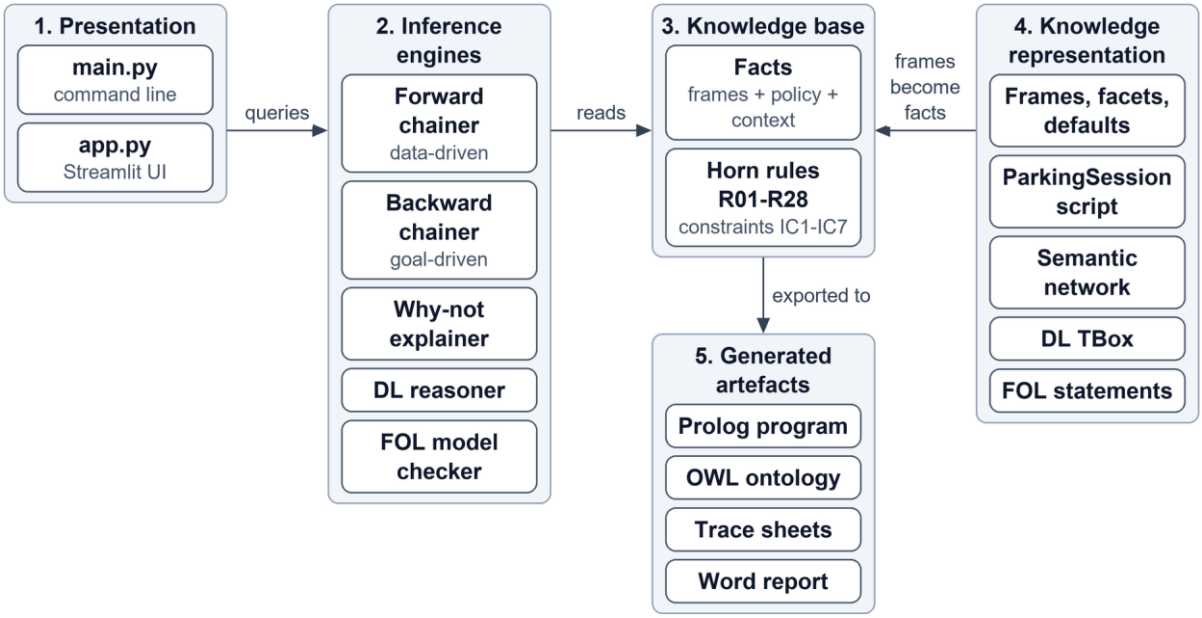


Figure 1: System architecture and knowledge model

Knowledge enters through the frames. The frame system settles every default and writes one fact per slot value, so the Horn knowledge base never re-types a fact. The same predicate names are used in the semantic network, FOL, DL and the Prolog export, and a unit test (tests/test_consistency.py) fails if any layer uses a name the others do not.

Table 2: Role of each paradigm in the project

Paradigm	Purpose in this project	Example
Frames and script	Concepts with slots, facets and inherited defaults; the stages of a parking visit	AccessibleSpace overrides spaceType = accessible
Semantic network	Visual map of concepts and relations, with inheritance and path queries	ParkingSpace part-of ParkingZone
FOL	Exact statement of rules and constraints with quantifiers	∀s (parking_space(s) → ∃z (parking_zone(z) ∧ located_in(s, z)))
Description logic	Classes, roles, subsumption, restrictions and disjointness; consistency checking	AuthorizedStudent ≡ Student ⊓ ∃hasPermit.(ValidPermit ⊓ StudentPermit)
Horn clauses	The executable policy, run by forward and backward chaining	`authorized(X, P) :- permit_matches_role(X, P), permit_valid(P).`

3.1 Frames

3.1.1 Frame architecture

A frame is a named structure with slots. Class frames describe a kind of thing and instance frames describe one individual; each has an ISA link to its parent and inherits all of its ancestors' slots. Each slot carries facets: type (str, int, symbol or frame:Class for a link), default, range (allowed values), cardinality, an if-needed procedure, the logic predicate it becomes and the network relation it becomes.

When a slot of an instance is read, the system looks in three places in order: the instance's own value, an if-needed procedure on the class or an ancestor, and the nearest default up the ISA chain. A subclass can override a default. ParkingSpace says spaceType is standard and AccessibleSpace says accessible, so s_a3 gets accessible without storing it. Logic cannot override a conclusion, so the frame layer settles

defaults first and hands one definite value per slot to the logic layer. This is how accessibility_badge(ali, no) reaches the knowledge base without anyone typing it. The 38 instance frames produce 135 facts, and FrameSystem.validate() checks every value against its type, range and cardinality facets.

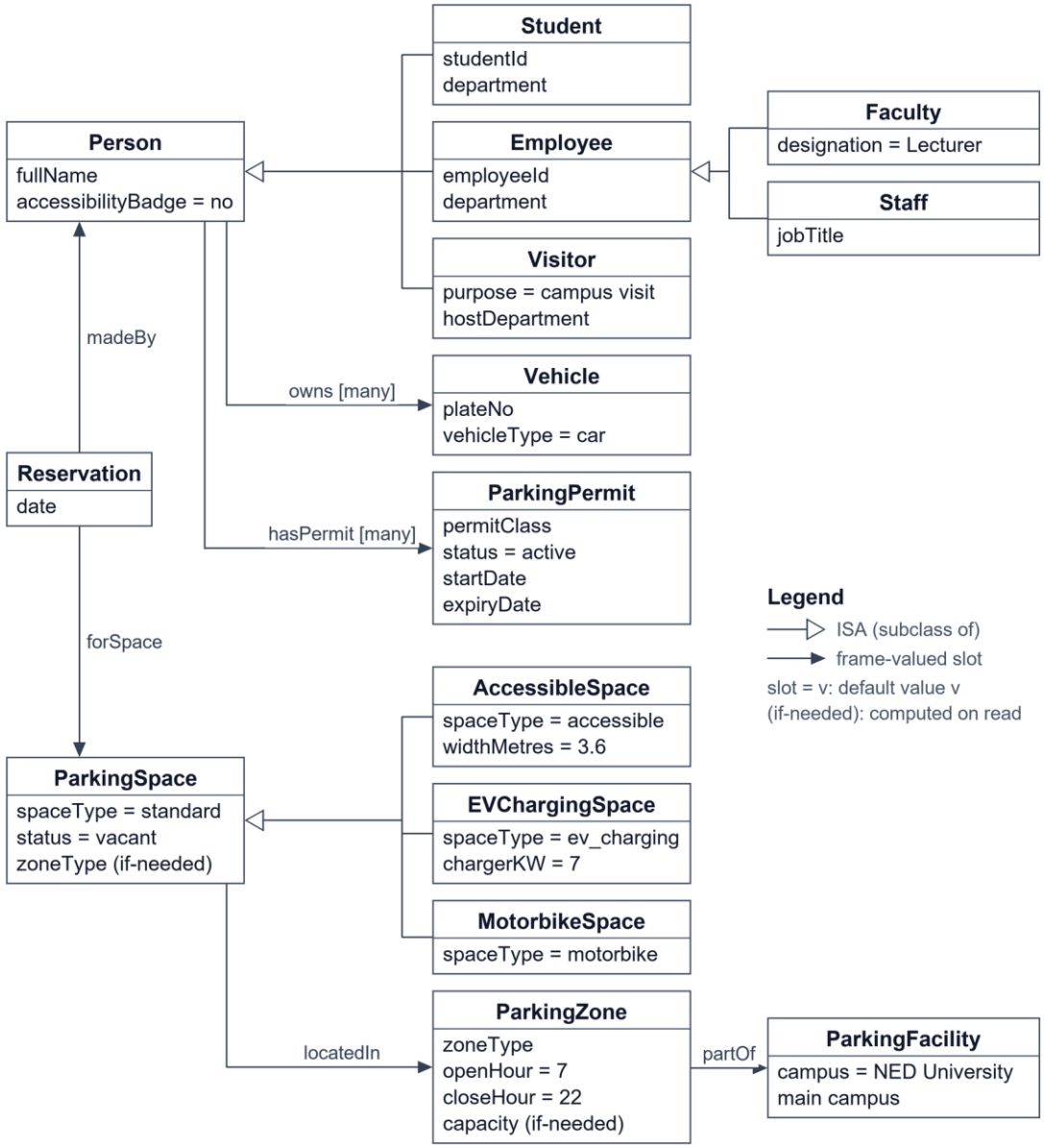


Figure 2: Frame hierarchy with slots and links

3.1.2 Frame: Person

Person is the root of the people hierarchy. Its four slots are inherited by every subclass. accessibilityBadge has the default no, so only Sara needs an explicit value.

Table 3: Frame Person

Slot	Type	Default	Other facets	Predicate
fullName	str		-	-
accessibilityBadge	symbol	no	range {yes, no}	accessibility_badge
owns	frame:Vehicle		multiple	owns
hasPermit	frame:ParkingPermit		multiple	has_permit

3.1.3 Frame: Student

Student inherits fullName, accessibilityBadge, owns and hasPermit from Person and adds two slots of its own.

Table 4: Frame Student (own slots)

Slot	Type	Default	Other facets	Predicate
studentId	str		-	-
department	str		-	-

Instances: ali, sara, usman, zara.

3.1.4 Frame: Faculty / Staff

Faculty and Staff both sit under Employee, which carries employeeId and department so that the employee rules apply to both. Faculty has the default designation Lecturer, which Dr. Ahmed overrides with Assistant Professor.

Table 5: Frames Employee, Faculty and Staff

Slot	Type	Default	Other facets	Predicate
Employee.employeeId	str		-	-
Employee.department	str		-	-
Faculty.designation	str	Lecturer	-	-
Staff.jobTitle	str		-	-

Instances: ahmed, fatima.

3.1.5 Frame: Visitor

Table 6: Frame Visitor (own slots)

Slot	Type	Default	Other facets	Predicate
purpose	str	campus visit	-	-
hostDepartment	str		-	-

Instances: bilal, hina.

A visitor has no permit by default. Bilal's hasPermit slot is filled with the one-day pass vp_bilal; Hina's is empty, which is why R09 cannot fire for her.

3.1.6 Frame: Vehicle

Table 7: Frame Vehicle

Slot	Type	Default	Other facets	Predicate
plateNo	str		-	-
vehicleType	symbol	car	range {car, electric_car, motorbike}	vehicle_type

Instances: car_ali, car_sara, car_usman, car_zara, car_bilal, car_hina, ev_ahmed, bike_fatima.

Six of the eight vehicles take the default car; ev_ahmed and bike_fatima set their own type.

3.1.7 Frame: Parking permit

Table 8: Frame ParkingPermit

Slot	Type	Default	Other facets	Predicate
permitClass	symbol		range {student_permit, employee_permit, visitor_pass}	permit_class
status	symbol	active	range {active, suspended, revoked}	permit_status
startDate	int		-	permit_start
expiryDate	int		-	permit_expiry

Instances: p_ali, p_sara, p_usman, p_zara, p_ahmed, p_fatima, vp_bilal.

Every permit has a start date and an expiry date, which R06 compares with today's date.

3.1.8 Frame: Parking space

ParkingSpace has the if-needed slot zoneType, which follows locatedIn to the zone and reads that zone's type. The three subclasses override the spaceType default and add their own slots.

Table 9: Frames ParkingSpace and subclasses

Slot	Type	Default	Other facets	Predicate
ParkingSpace.locatedIn	frame:ParkingZone		-	located_in
ParkingSpace.spaceType	symbol	standard	range {standard, accessible, ev_charging, motorbike}	space_type
ParkingSpace.status	symbol	vacant	range {vacant, occupied, reserved}	space_status
ParkingSpace.zoneType	symbol		if-needed: zoneType of the zone this space is located in	-
AccessibleSpace.spaceType	symbol	accessible	overrides ParkingSpace; range {standard, accessible, ev_charging, motorbike}	space_type
AccessibleSpace.widthMetres	str	3.6	-	-
EVChargingSpace.spaceType	symbol	ev_charging	overrides ParkingSpace; range {standard, accessible, ev_charging, motorbike}	space_type
EVChargingSpace.chargerKW	int	7	-	-
MotorbikeSpace.spaceType	symbol	motorbike	overrides ParkingSpace; range {standard, accessible, ev_charging, motorbike}	space_type

Instances: s_a1, s_a2, s_a3, s_a4, s_a5, s_e1, s_e2, s_e3, s_v1.

3.1.9 Frame: Parking zone

capacity is computed on demand: it counts the bays whose locatedIn is this zone, so zone_a gives 5 and nobody has to keep the number up to date.

Table 10: Frame ParkingZone

Slot	Type	Default	Other facets	Predicate
zoneType	symbol		range {student_zone, employee_zone, visitor_zone}	zone_type
openHour	int	7	-	zone_opens
closeHour	int	22	-	zone_closes
partOf	frame:ParkingFacility		-	part_of
capacity	int		if-needed: count of ParkingSpace instances whose locatedIn is this zone	-

Instances: zone_a, zone_e, zone_v.

3.1.10 Frame: Reservation

A reservation links three things (person, bay, date), so it is a frame of its own and not a slot on Person. Its three slots become the binary predicates reserved_by, reserved_space and reservation_date.

Table 11: Frame Reservation

Slot	Type	Default	Other facets	Predicate
madeBy	frame:Person		-	reserved_by
forSpace	frame:ParkingSpace		-	reserved_space
date	int		-	reservation_date

Instances: r1, r2.

3.1.11 Script / event sequence

A frame describes a thing; a script describes a stereotyped sequence of events (Schank and Abelson, 1977). ParkingSession has the usual parts: track (campus parking with a permit or pass), roles (driver, gate system, parking office), props (vehicle, permit, zone, bay, reservation), entry conditions (a registered vehicle, a request, a known date and hour) and results (bay occupied, driver parked). Its seven scenes (Figure 3) are arrive at the gate, permit check, zone check, bay check, availability, allocation and park. Together they cover the whole request: validate, check eligibility, find a suitable bay, resolve conflicts, and allocate or deny.

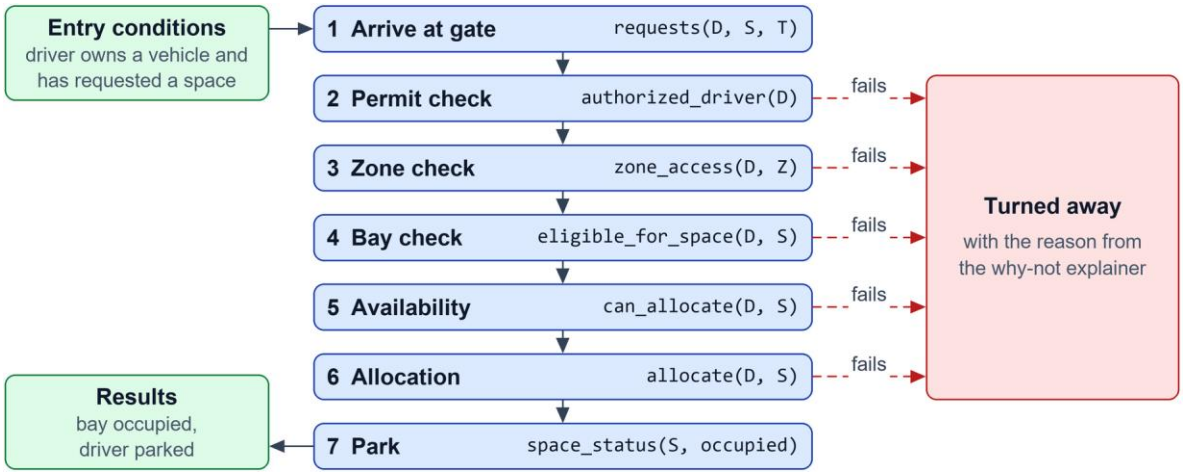


Figure 3: ParkingSession script scenes

The script runs. Each scene asks the backward chainer one goal and the script stops at the first scene that fails, with the reason from the explainer:

```
Running script ParkingSession for driver=usman, space=s_a1
  Scene 1 Arrive at gate   PASS  requests(usman, s_a1, T)
  Scene 2 Permit check     STOP  authorized_driver(usman)
      reason: permit_valid(p_usman) fails at 20260915 >= 20261002 (false)
```

For Fatima and s_e1 the script gets one scene further and stops at the bay check, because her motorbike does not fit a standard bay. For Sara and s_a1 all six checks pass and scene 7 changes the bay's status to occupied.

3.2 Semantic Networks

3.2.1 Semantic network design

The network is generated from the frames: every class frame is a node, every ISA link an is-a edge and every frame-valued slot a labelled edge. Because it is generated, its labels always match the frame and rule vocabulary. The first figure below shows the concept level. The second shows one worked example: Ali, his car and permit, Zara's reservation r1 and the bay, zone and values on the paths in section 3.2.3.

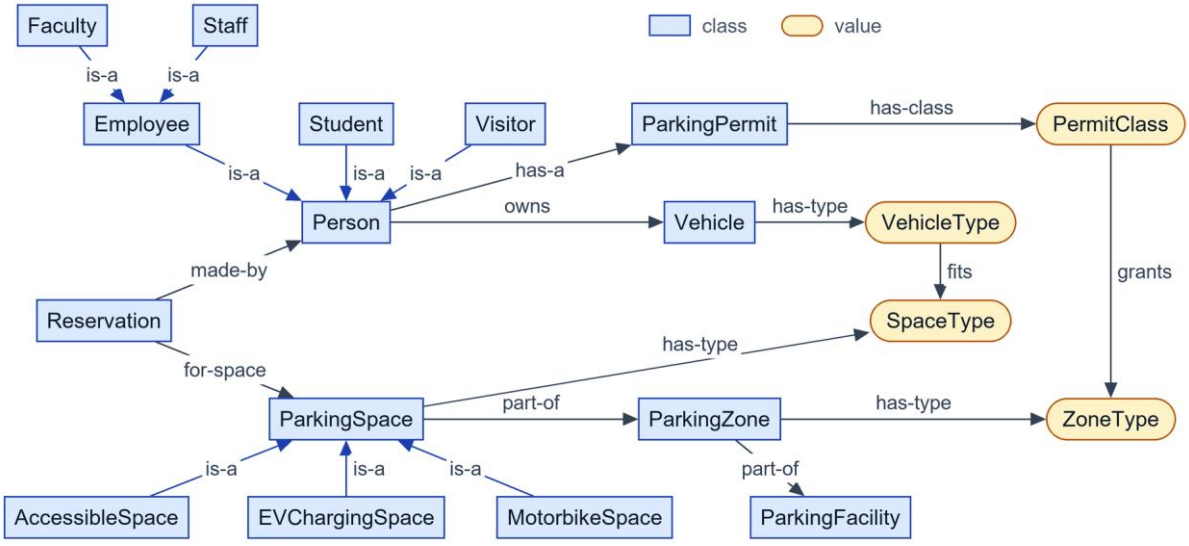


Figure 4: Concept-level semantic network

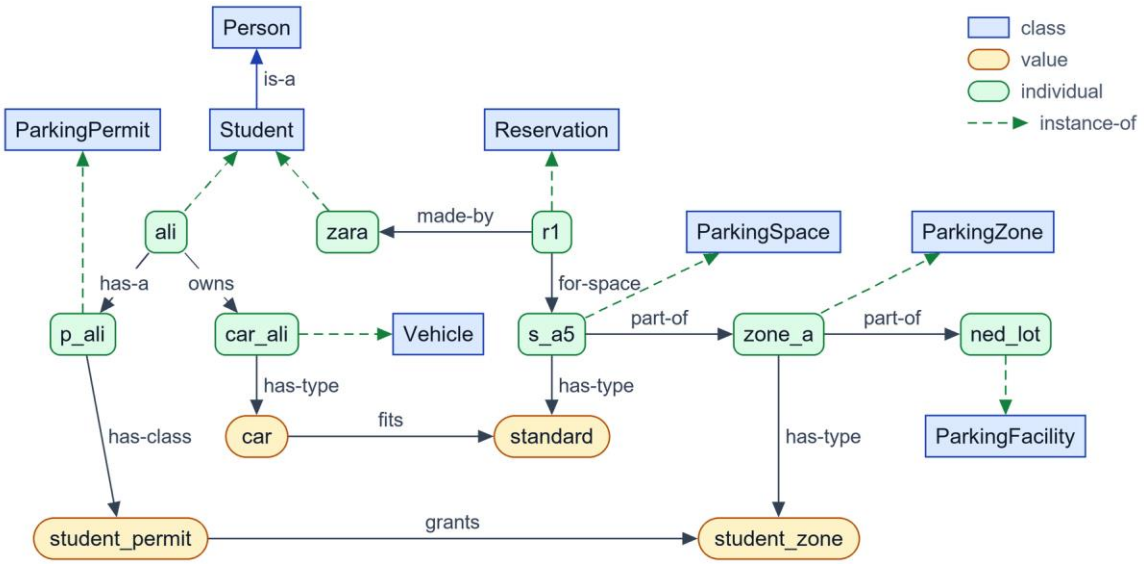


Figure 5: Semantic network with individuals

3.2.2 Relationship types

Table 12: Relationship types in the semantic network

Relationship	Meaning	Example
is-a	Subclass link; the child inherits every relation of the parent	Student is-a Person
instance-of	Individual to its class	ali instance-of Student
has-a	A person holds a permit	Person has-a ParkingPermit
owns	A person owns a registered vehicle	Person owns Vehicle
part-of (located-in)	Physical containment	ParkingSpace part-of ParkingZone part-of ParkingFacility
made-by, for-space (reserves)	The two sides of a reservation	Reservation made-by Person; Reservation for-space ParkingSpace
has-type, has-class	Entity to a value concept	ParkingSpace has-type SpaceType
grants, fits	Policy between value concepts	PermitClass grants ZoneType; VehicleType fits SpaceType
eligible-for (derived)	Produced by the rules, not stored	ali eligible_for_space s_a1

3.2.3 Semantic network walkthrough

The important links in the network, read as paths:

1. Faculty to Person: Faculty is-a Employee is-a Person. Faculty has no owns edge of its own, but inherits "owns Vehicle" along this path, which is how Dr. Ahmed can own ev_ahmed.
2. Bay to facility: s_a1 instance-of ParkingSpace, part-of zone_a, part-of ned_lot. This is the containment chain the DL axioms T15 and T16 require.
3. Permit to zone: p_ali has-class student_permit, which grants student_zone, which is the has-type of zone_a. This path is rule R13 drawn as a graph.
4. Vehicle to bay: car_ali has-type car, which fits standard, which is the has-type of s_a1. This path is the fit test in R14; for Fatima the path motorbike fits standard does not exist, which is why TC6 fails.
5. Reservation: r1 made-by zara and r1 for-space s_a5. Ali has no made-by edge into any reservation for s_a5, so R20 cannot allocate it to him (TC11).
6. Path query: the shortest chain from Faculty to ParkingZone found by the network code is Faculty is-a Employee is-a Person, made-by from Reservation, for-space ParkingSpace, part-of ParkingZone.

4. FORMAL LOGIC REPRESENTATION

4.1 First-Order Logic (FOL)

4.1.1 Predicates and constants

- Constants name individuals and values: ali, p_ali, s_a1, zone_a, car, accessible, 20261002.

- Variables are lower case in formulas (x, s, p) and upper case in Horn clauses (X, S, P).
- Unary predicates give class membership: person, student, employee, faculty, staff, visitor, vehicle, permit, parking_zone, parking_space, reservation.
- Binary predicates give attributes and relations: owns, has_permit, permit_class, permit_status, permit_start, permit_expiry, vehicle_type, located_in, part_of, space_type, space_status, zone_type, zone_opens, zone_closes, reserved_by, reserved_space, reservation_date, accessibility_badge, grants, fits.
- Context predicates: current_date(d), current_hour(h), requests(x, s, t).
- Derived predicates: permit_valid, permit_matches_role, authorized, authorized_driver, zone_open, zone_access, suitable_space, eligible_for_space, space_free, holds_reservation, can_allocate, priority, loses, double_booked, allocate.
- Connectives and quantifiers: ¬, ∧, ∨, →, ∀, ∃, =, and the comparisons ≥, ≤, >, < on dates and hours stored as integers. No function symbols are used.

4.1.2 Domain facts

Facts are ground atoms. Some examples from the knowledge base, with their meaning:

- student(ali): Ali is a student.
- has_permit(ali, p_ali) ∧ permit_class(p_ali, student_permit): Ali holds a student permit.
- permit_expiry(p_usman, 20260915): Usman's permit expired on 15 September 2026.
- permit_start(vp_bilal, 20261002) ∧ permit_expiry(vp_bilal, 20261002): Bilal's visitor pass is valid on 2 October 2026 only.
- requests(sara, s_a1, 910): Sara asked for s_a1 at 09:10.

4.1.3 FOL rules

A Horn rule head :- b1, ..., bn is read as ∀x1...∀xk ((b1 ∧ ... ∧ bn) → head), where x1...xk are all its variables. The table below gives ten of the 28 rules; the FOL column is printed by Rule.to_fol() from the same rule objects the engines run. All 28 are listed as Horn clauses in section 5.3.

Table 13: FOL form of selected rules

ID	Natural-language rule	FOL formula	Explanation
FOL-01 (R04)	Every faculty member is an employee.	∀x (faculty(x) → employee(x))	Horn form of Faculty ⊑ Employee. Faculty and staff share the employee rules.
FOL-02 (R06)	A permit is valid if its status is active and today lies between its start date and its expiry date.	∀p ∀b ∀e ∀d ((permit_status(p, active) ∧ permit_start(p, b) ∧ permit_expiry(p, e) ∧ current_date(d) ∧ b ≤ d ∧ e ≥ d) → permit_valid(p))	Expired, not-yet-started or suspended permits never satisfy this rule, so they can never authorize parking. B =< D and E >= D are built-in comparisons on YYYYMMDD integers. A one-day visitor pass has B = E.
FOL-03 (R07)	A student permit only counts when its holder is a student.	∀x ∀p ((has_permit(x, p) ∧ permit_class(p, student_permit) ∧ student(x)) → permit_matches_role(x, p))	Stops a permit being used by someone of the wrong category.
FOL-04 (R10)	A person is authorized by a permit that matches their role and is valid.	∀x ∀p ((permit_matches_role(x, p) ∧ permit_valid(p)) → authorized(x, p))	Joins the role check (R07-R09) with the validity check (R06).
FOL-05 (R11)	Anyone authorized by at least one permit is an authorized driver.	∀x ∀p (authorized(x, p) → authorized_driver(x))	P occurs only in the body, so logically it is existential: ∀x (∃p authorized(x, p) → authorized_driver(x)).
FOL-06 (R13)	An authorized person may enter a zone if their permit class grants that zone type and the zone is open.	∀x ∀z ∀p ∀c ∀t ((authorized(x, p) ∧ permit_class(p, c) ∧ grants(c, t) ∧ zone_type(z, t) ∧ zone_open(z)) → zone_access(x, z))	Students reach only student zones, employees only employee zones, visitors only the visitor zone, as listed in the grants/2 policy table.
FOL-07 (R14)	A space suits a person if it is in a zone they may enter and their vehicle fits the bay.	∀x ∀s ∀z ∀v ∀vt ∀st ((zone_access(x, z) ∧ located_in(s, z) ∧ owns(x, v) ∧ vehicle_type(v, vt) ∧ space_type(s, st) ∧ fits(vt, st)) → suitable_space(x, s))	A motorbike never fits a car bay and a car never fits a motorbike bay (fits/2 table).
FOL-08 (R16)	An accessible bay is only for a person who holds an accessibility badge.	∀x ∀s ((suitable_space(x, s) ∧ space_type(s, accessible) ∧ accessibility_badge(x, yes)) → eligible_for_space(x, s))	Together R15 and R16 cover every bay type; an accessible bay has no rule that ignores the badge.

ID	Natural-language rule	FOL formula	Explanation
FOL-09 (R19)	A free space can be allocated to any person eligible for it.	∀x ∀s ((eligible_for_space(x, s) ∧ space_free(s)) → can_allocate(x, s))	Occupied spaces never satisfy space_free, so they are never allocated.
FOL-10 (R25)	A requester loses a space to another eligible requester with higher priority.	∀y ∀s ∀ty ∀x ∀tx ∀px ∀py ((requests(y, s, ty) ∧ requests(x, s, tx) ∧ x ≠ y ∧ can_allocate(x, s) ∧ priority(x, px) ∧ priority(y, py) ∧ px > py) → loses(y, s))	Only competitors who could actually get the space (can_allocate) count. An ineligible requester cannot block anyone.

4.1.4 Quantifier usage

Every rule is universally quantified over all its variables, and the scope of each quantifier is the whole implication. A variable that appears only in the body can equally be read as existential inside the antecedent. R11 is the clearest case: ∀x ∀p (authorized(x, p) → authorized_driver(x)) is equivalent to ∀x (∃p authorized(x, p) → authorized_driver(x)), "anyone authorized by some permit is an authorized driver".

Some knowledge cannot be written as a Horn rule because it needs ∃ in the conclusion, disjunction, equality or classical negation. The next table lists twelve such statements. The model checker evaluates each one on the model built by forward chaining (domain: all 81 constants) and returns a counterexample if one fails.

Table 14: Further FOL statements

ID	Formula	Meaning	Model check
F01	∀s (parking_space(s) → ∃z (parking_zone(z) ∧ located_in(s, z)))	Every parking space lies in some parking zone.	holds
F02	∀s ∀z1 ∀z2 ((located_in(s, z1) ∧ located_in(s, z2)) → z1 = z2)	A space lies in at most one zone (located_in is functional).	holds
F03	∀x (person(x) → (student(x) ∨ employee(x) ∨ visitor(x)))	Every person is a student, an employee or a visitor.	holds
F04	∀x ¬(student(x) ∧ employee(x))	Nobody is both a student and an employee.	holds
F05	∀x ∀y ∀s ((allocate(x, s) ∧ allocate(y, s)) → x = y)	A space is allocated to at most one person.	holds
F06	∀x ∀s (allocate(x, s) → ¬space_status(s, occupied))	An occupied space is never allocated.	holds
F07	∀x ∀s ((allocate(x, s) ∧ space_type(s, accessible)) → accessibility_badge(x, yes))	Whoever is allocated an accessible bay holds a badge.	holds
F08	∀x (authorized_driver(x) → ∃p (has_permit(x, p) ∧ permit_valid(p)))	Every authorized driver holds at least one valid permit.	holds
F09	∃s (located_in(s, zone_a) ∧ space_free(s))	There is at least one free space in the student zone.	holds
F10	∀x ((visitor(x) ∧ ¬∃p has_permit(x, p)) → ¬authorized_driver(x))	A visitor with no permit or pass is not an authorized driver.	holds
F11	∀x ∀s (allocate(x, s) → ∃t requests(x, s, t))	Every allocation answers a request.	holds
F12	∀p (permit_valid(p) → ∃b ∃e ∃d (permit_start(p, b) ∧ permit_expiry(p, e) ∧ current_date(d) ∧ b ≤ d ∧ e ≥ d))	A valid permit is in force today: it has started and has not expired.	holds

- F01 has the pattern ∀s (... → ∃z ...): for every bay there is a zone. Swapping the quantifiers to ∃z ∀s would claim that one zone holds every bay, which is false.
- F05 needs three universal variables because it compares two allocations of the same bay.
- F09 is purely existential. Backward chaining answers it with the query located_in(S, zone_a), space_free(S) and returns S = s_a1.
- F10 has ¬∃p inside the antecedent. It holds in our model but does not follow from the rules in classical FOL; it needs the closed-world assumption (section 5.4).

4.1.5 Consistency verification

Each formula was checked in four ways. Every variable is bound by exactly one quantifier whose scope is the whole formula. Every head variable of a rule also appears in a positive body literal, and every variable in a comparison or under "not" is bound earlier (range restriction, enforced by Rule.check_safety() when the rule is parsed). Every implication points from the conditions to the conclusion. Finally the statements are evaluated on real data: all twelve hold on the main scenario, and F04 fails on the TC12 data with the counterexample x = ali, which is the expected result.

4.2 Description Logic (DL)

The language is ALCI: ⊓, ⊔, ∃R.C, ∀R.C, inverse roles R^{-} and ⊥.

4.2.1 DL concepts

Atomic concepts: Person, Student, Employee, Faculty, Staff, Visitor, BadgeHolder, NonBadgeHolder, Vehicle, Car, ElectricCar, Motorbike, ParkingPermit, StudentPermit, EmployeePermit, VisitorPass, ValidPermit, ParkingFacility, ParkingZone, ParkingSpace, AccessibleSpace, EVChargingSpace, MotorbikeSpace and Reservation. Defined concepts: ValidPermitHolder, AuthorizedStudent, AuthorizedEmployee, AuthorizedVisitor and AuthorizedDriver.

4.2.2 DL roles

Each role is one binary predicate of the knowledge base, so the ABox is read straight from the facts.

Table 15: DL roles

Role	Domain → range	Horn predicate
owns	Person → Vehicle	owns
hasPermit	Person → ParkingPermit	has_permit
locatedIn	ParkingSpace → ParkingZone	located_in
partOf	ParkingZone → ParkingFacility	part_of
madeBy	Reservation → Person	reserved_by
forSpace	Reservation → ParkingSpace	reserved_space
allocatedTo	ParkingSpace → Person	allocate

4.2.3 Subsumption axioms

C ⊑ D says every C is a D. These axioms build the hierarchy and are the same knowledge as Horn rules R01-R05.

Table 16: Subsumption axioms

Axiom	Axiom	Axiom
T01 Student ⊑ Person	T02 Employee ⊑ Person	T03 Faculty ⊑ Employee
T04 Staff ⊑ Employee	T05 Visitor ⊑ Person	T06 BadgeHolder ⊑ Person
T07 NonBadgeHolder ⊑ Person	T08 StudentPermit ⊑ ParkingPermit	T09 EmployeePermit ⊑ ParkingPermit
T10 VisitorPass ⊑ ParkingPermit	T11 ValidPermit ⊑ ParkingPermit	T12 AccessibleSpace ⊑ ParkingSpace
T13 EVChargingSpace ⊑ ParkingSpace	T14 MotorbikeSpace ⊑ ParkingSpace	T21 Motorbike ⊑ Vehicle
T22 ElectricCar ⊑ Vehicle	T23 Car ⊑ Vehicle	

4.2.4 Disjointness axioms

C ⊓ D ⊑ ⊥ says no individual is in both. Disjointness is inherited: D01 makes Faculty and Student disjoint too, because Faculty ⊑ Employee.

Table 17: Disjointness axioms

Axiom	Axiom
D01 Student ⊓ Employee ⊑ ⊥	D02 Student ⊓ Visitor ⊑ ⊥
D03 Employee ⊓ Visitor ⊑ ⊥	D04 Faculty ⊓ Staff ⊑ ⊥
D05 BadgeHolder ⊓ NonBadgeHolder ⊑ ⊥	D06 StudentPermit ⊓ EmployeePermit ⊑ ⊥
D07 StudentPermit ⊓ VisitorPass ⊑ ⊥	D08 EmployeePermit ⊓ VisitorPass ⊑ ⊥
D09 AccessibleSpace ⊓ EVChargingSpace ⊑ ⊥	D10 AccessibleSpace ⊓ MotorbikeSpace ⊑ ⊥
D11 EVChargingSpace ⊓ MotorbikeSpace ⊑ ⊥	D12 Motorbike ⊓ ElectricCar ⊑ ⊥
D13 Motorbike ⊓ Car ⊑ ⊥	D14 Car ⊓ ElectricCar ⊑ ⊥
D15 Person ⊓ Vehicle ⊑ ⊥	D16 Person ⊓ ParkingSpace ⊑ ⊥
D17 ParkingSpace ⊓ ParkingZone ⊑ ⊥	

4.2.5 Membership assertions

ABox concept assertions come from the frames and from derived facts; role assertions come from the binary facts. The main scenario has 114 assertions, for example Student(ali), Faculty(ahmed), NonBadgeHolder(ali), BadgeHolder(sara), AccessibleSpace(s_a3), ValidPermit(p_ali), StudentPermit(p_ali), hasPermit(ali, p_ali), owns(ahmed, ev_ahmed), locatedIn(s_a1, zone_a), partOf(zone_a, ned_lot), madeBy(r1, zara) and forSpace(r1, s_a5).

4.2.6 Existential and universal restrictions

Table 18: Restrictions

ID	Axiom	Meaning
T15	ParkingSpace ⊑ ∃locatedIn.ParkingZone	Every parking space lies in some parking zone (existential restriction).
T16	ParkingZone ⊑ ∃partOf.ParkingFacility	Every zone is part of some parking facility.
T17	Reservation ⊑ ∃madeBy.Person ⊓ ∃forSpace.ParkingSpace	Every reservation is made by some person and is for some parking space.
T18	Vehicle ⊑ ∃owns⁻.Person	Every vehicle is owned by some person (owns⁻ is the inverse of owns).
T19	AccessibleSpace ⊑ ∀allocatedTo.BadgeHolder	An accessible bay may only be allocated to badge holders (universal restriction).
T20	MotorbikeSpace ⊑ ∀allocatedTo.∃owns.Motorbike	A motorbike bay may only be allocated to someone who owns a motorbike.

Table 19: Defined concepts

ID	Axiom	Meaning
T24	ValidPermitHolder ≡ Person ⊓ ∃hasPermit.ValidPermit	A valid-permit holder is exactly a person with at least one valid permit.
T25	AuthorizedStudent ≡ Student ⊓ ∃hasPermit.(ValidPermit ⊓ StudentPermit)	An authorized student is a student holding a valid student permit.
T26	AuthorizedEmployee ≡ Employee ⊓ ∃hasPermit.(ValidPermit ⊓ EmployeePermit)	An authorized employee is an employee holding a valid employee permit.
T27	AuthorizedVisitor ≡ Visitor ⊓ ∃hasPermit.(ValidPermit ⊓ VisitorPass)	An authorized visitor is a visitor holding a valid visitor pass.
T28	AuthorizedDriver ≡ AuthorizedStudent ⊔ AuthorizedEmployee ⊔ AuthorizedVisitor	An authorized driver is an authorized student, employee or visitor (matches Horn rules R07-R11).

4.2.7 DL interpretation

The ∃ axioms (T15-T18) say that something must exist: every bay is in some zone, every vehicle has some owner. They do not name it. The ∀ axioms (T19, T20) restrict whatever fillers exist: anyone an accessible bay is allocated to must be a badge holder. The definitions (T24-T28) work in both directions, so the reasoner can recognise an AuthorizedStudent from her permit, not only list her.

The reasoner classifies the TBox and finds 13 subsumptions that nobody wrote, such as AuthorizedStudent ⊑ AuthorizedDriver (from T25 and T28). Realization agrees with the Horn rules: an individual is an AuthorizedDriver in DL exactly when authorized_driver holds after forward chaining, and a unit test checks this for all eight people. T19 is active too: if a test forces s_a3 to be allocated to Ali, ∀-propagation adds BadgeHolder(ali), which clashes with NonBadgeHolder(ali) through D05, and the reasoner reports it. In TC12 the reasoner reports "ali is both Employee and Student, which D01 declares disjoint".

The DL reasoner uses the open-world assumption. Usman holds a permit, but nothing says it is a ValidPermit, so the reasoner cannot conclude AuthorizedDriver(usman), and it cannot conclude the negation either; the answer is "unknown". The Horn engine uses the closed-world assumption, so authorized_driver(usman) is simply false and Usman is turned away. The gate needs the closed-world answer; a shared ontology needs the open-world reading, because a missing fact there usually means "not recorded".

5. HORN CLAUSE KNOWLEDGE BASE

5.1 Knowledge base

The knowledge base of the main scenario has 164 base facts: 142 from the frames, 12 policy facts (grants/2, fits/2, open_type/1) and 10 context facts (date, hour and eight requests). Forward chaining adds 105 derived facts such as permit_valid, zone_access, priority and allocate. Base facts are never changed by the rules; derived facts are recomputed on every run, so a change in the data always gives a fresh, consistent result.

5.2 Facts

A fact is a Horn clause with no body. All 164 facts of the main scenario, in groups:

People and roles (16): `student(ali); student(sara); student(usman); student(zara); faculty(ahmed); staff(fatima); visitor(bilal); visitor(hina); accessibility_badge(ahmed, no); accessibility_badge(ali, no); accessibility_badge(bilal, no); accessibility_badge(fatima, no); accessibility_badge(hina, no); accessibility_badge(sara, yes); accessibility_badge(usman, no); accessibility_badge(zara, no).`

Vehicles and ownership (24): `vehicle(bike_fatima); vehicle(car_ali); vehicle(car_bilal); vehicle(car_hina); vehicle(car_sara); vehicle(car_usman); vehicle(car_zara); vehicle(ev_ahmed); owns(ahmed, ev_ahmed); owns(ali, car_ali); owns(bilal, car_bilal); owns(fatima, bike_fatima); owns(hina, car_hina); owns(sara, car_sara); owns(usman, car_usman); owns(zara, car_zara); vehicle_type(bike_fatima, motorbike); vehicle_type(car_ali, car); vehicle_type(car_bilal, car); vehicle_type(car_hina, car); vehicle_type(car_sara, car); vehicle_type(car_usman, car); vehicle_type(car_zara, car); vehicle_type(ev_ahmed, electric_car).`

Permits (42): `permit(p_ahmed); permit(p_ali); permit(p_fatima); permit(p_sara); permit(p_usman); permit(p_zara); permit(vp_bilal); has_permit(ahmed, p_ahmed); has_permit(ali, p_ali); has_permit(bilal, vp_bilal); has_permit(fatima, p_fatima); has_permit(sara, p_sara); has_permit(usman, p_usman); has_permit(zara, p_zara); permit_class(p_ahmed, employee_permit); permit_class(p_ali, student_permit); permit_class(p_fatima, employee_permit); permit_class(p_sara, student_permit); permit_class(p_usman, student_permit); permit_class(p_zara, student_permit); permit_class(vp_bilal, visitor_pass); permit_status(p_ahmed, active); permit_status(p_ali, active); permit_status(p_fatima, active); permit_status(p_sara, active); permit_status(p_usman, active); permit_status(p_zara, active); permit_status(vp_bilal, active); permit_start(p_ahmed, 20260101); permit_start(p_ali, 20260701); permit_start(p_fatima, 20260101); permit_start(p_sara, 20260701); permit_start(p_usman, 20250916); permit_start(p_zara, 20260701); permit_start(vp_bilal, 20261002); permit_expiry(p_ahmed, 20271231); permit_expiry(p_ali, 20270630); permit_expiry(p_fatima, 20271231); permit_expiry(p_sara, 20270630); permit_expiry(p_usman, 20260915); permit_expiry(p_zara, 20270630); permit_expiry(vp_bilal, 20261002).`

Facility and zones (16): `parking_facility(ned_lot); parking_zone(zone_a); parking_zone(zone_e); parking_zone(zone_v); part_of(zone_a, ned_lot); part_of(zone_e, ned_lot); part_of(zone_v, ned_lot); zone_type(zone_a, student_zone); zone_type(zone_e, employee_zone); zone_type(zone_v, visitor_zone); zone_opens(zone_a, 7); zone_opens(zone_e, 6); zone_opens(zone_v, 8); zone_closes(zone_a, 22); zone_closes(zone_e, 23); zone_closes(zone_v, 17).`

Parking spaces (36): `parking_space(s_a1); parking_space(s_a2); parking_space(s_a3); parking_space(s_a4); parking_space(s_a5); parking_space(s_e1); parking_space(s_e2); parking_space(s_e3); parking_space(s_v1); located_in(s_a1, zone_a); located_in(s_a2, zone_a); located_in(s_a3, zone_a); located_in(s_a4, zone_a); located_in(s_a5, zone_a); located_in(s_e1, zone_e); located_in(s_e2, zone_e); located_in(s_e3, zone_e); located_in(s_v1, zone_v); space_type(s_a1, standard); space_type(s_a2, standard); space_type(s_a3, accessible); space_type(s_a4, motorbike); space_type(s_a5, standard); space_type(s_e1, standard); space_type(s_e2, ev_charging); space_type(s_e3, standard); space_type(s_v1, standard); space_status(s_a1, vacant); space_status(s_a2, occupied); space_status(s_a3, vacant); space_status(s_a4, vacant); space_status(s_a5, reserved); space_status(s_e1, vacant); space_status(s_e2, vacant); space_status(s_e3, reserved); space_status(s_v1, vacant).`

Reservations (8): `reservation(r1); reservation(r2); reserved_by(r1, zara); reserved_by(r2, ahmed); reserved_space(r1, s_a5); reserved_space(r2, s_e3); reservation_date(r1, 20261002); reservation_date(r2, 20261002).`

Policy tables (12): `grants(employee_permit, employee_zone); grants(student_permit, student_zone); grants(visitor_pass, visitor_zone); fits(car, accessible); fits(car, standard); fits(electric_car, accessible); fits(electric_car, ev_charging); fits(electric_car, standard); fits(motorbike, motorbike); open_type(ev_charging); open_type(motorbike); open_type(standard).`

Context (per scenario) (10): `current_date(20261002); current_hour(10); requests(ahmed, s_e3, 800); requests(ali, s_a1, 905); requests(bilal, s_v1, 920); requests(fatima, s_e1, 830); requests(hina, s_v1, 915); requests(sara, s_a1, 910); requests(usman, s_a1, 900); requests(zara, s_a1, 850).`

5.3 Rules

A rule is a definite Horn clause: exactly one positive literal (the head) and one or more body literals. The 28 rules fall into eight groups: class hierarchy (R01-R05), permits and authorization (R06-R11), zones and time (R12-R13), suitability and eligibility (R14-R16), availability (R17-R20), priority (R21-R24), conflicts (R25-R27) and the decision (R28). A Horn clause with no positive literal is a denial, written `:- body.`, which says the body must never be true. IC1-IC7 are checked after every forward-chaining run.

Table 20: Horn rule base and constraints

ID	Natural language	Horn clause
R01	Every student is a person.	`person(X) :- student(X).`
R02	Every employee is a person.	`person(X) :- employee(X).`
R03	Every visitor is a person.	`person(X) :- visitor(X).`
R04	Every faculty member is an employee.	`employee(X) :- faculty(X).`
R05	Every staff member is an employee.	`employee(X) :- staff(X).`
R06	A permit is valid if its status is active and today lies between its start date and its expiry date.	`permit_valid(P) :- permit_status(P, active), permit_start(P, B), permit_expiry(P, E), current_date(D), B =< D, E >= D.`
R07	A student permit only counts when its holder is a student.	`permit_matches_role(X, P) :- has_permit(X, P), permit_class(P, student_permit), student(X).`
R08	An employee permit only counts when its holder is faculty or staff.	`permit_matches_role(X, P) :- has_permit(X, P), permit_class(P, employee_permit), employee(X).`
R09	A visitor needs a visitor pass; the pass is the visitor's authorization.	`permit_matches_role(X, P) :- has_permit(X, P), permit_class(P, visitor_pass), visitor(X).`

ID	Natural language	Horn clause
R10	A person is authorized by a permit that matches their role and is valid.	`authorized(X, P) :- permit_matches_role(X, P), permit_valid(P).`
R11	Anyone authorized by at least one permit is an authorized driver.	`authorized_driver(X) :- authorized(X, P).`
R12	A zone is open when the current hour is inside its opening hours.	`zone_open(Z) :- zone_opens(Z, O), zone_closes(Z, C), current_hour(H), H >= O, H < C.`
R13	An authorized person may enter a zone if their permit class grants that zone type and the zone is open.	`zone_access(X, Z) :- authorized(X, P), permit_class(P, C), grants(C, T), zone_type(Z, T), zone_open(Z).`
R14	A space suits a person if it is in a zone they may enter and their vehicle fits the bay.	`suitable_space(X, S) :- zone_access(X, Z), located_in(S, Z), owns(X, V), vehicle_type(V, VT), space_type(S, ST), fits(VT, ST).`
R15	A suitable standard, EV or motorbike bay needs nothing more.	`eligible_for_space(X, S) :- suitable_space(X, S), space_type(S, T), open_type(T).`
R16	An accessible bay is only for a person who holds an accessibility badge.	`eligible_for_space(X, S) :- suitable_space(X, S), space_type(S, accessible), accessibility_badge(X, yes).`
R17	A vacant space is free.	`space_free(S) :- space_status(S, vacant).`
R18	A person holds a reservation for a space if they made a reservation for it for today.	`holds_reservation(X, S) :- reserved_by(R, X), reserved_space(R, S), reservation_date(R, D), current_date(D).`
R19	A free space can be allocated to any person eligible for it.	`can_allocate(X, S) :- eligible_for_space(X, S), space_free(S).`
R20	A reserved space can be allocated only to the person holding today's reservation.	`can_allocate(X, S) :- eligible_for_space(X, S), space_status(S, reserved), holds_reservation(X, S).`
R21	Badge holders have priority 3 (highest).	`priority(X, 3) :- person(X), accessibility_badge(X, yes).`
R22	Faculty and staff without a badge have priority 2.	`priority(X, 2) :- employee(X), accessibility_badge(X, no).`
R23	Students without a badge have priority 1.	`priority(X, 1) :- student(X), accessibility_badge(X, no).`
R24	Visitors without a badge have priority 1.	`priority(X, 1) :- visitor(X), accessibility_badge(X, no).`
R25	A requester loses a space to another eligible requester with higher priority.	`loses(Y, S) :- requests(Y, S, TY), requests(X, S, TX), X \= Y, can_allocate(X, S), priority(X, PX), priority(Y, PY), PX > PY.`
R26	With equal priority, the earlier request wins (first come, first served).	`loses(Y, S) :- requests(Y, S, TY), requests(X, S, TX), X \= Y, can_allocate(X, S), priority(X, P), priority(Y, P), TX < TY.`
R27	A person holding two different reservations for the same day is double-booked.	`double_booked(X) :- reserved_by(R1, X), reserved_by(R2, X), R1 \= R2, reservation_date(R1, D), reservation_date(R2, D).`
R28	Allocate the requested space if it can be allocated, the requester loses no conflict for it and is not double-booked.	`allocate(X, S) :- requests(X, S, T), can_allocate(X, S), not loses(X, S), not double_booked(X).`
IC1	One space is never allocated to two people.	`:- allocate(X, S), allocate(Y, S), X \= Y.`
IC2	One person is never allocated two spaces.	`:- allocate(X, S1), allocate(X, S2), S1 \= S2.`
IC3	An occupied space is never allocated.	`:- allocate(X, S), space_status(S, occupied).`
IC4	Nobody is both a student and an employee (Student ⊓ Employee ⊑ ⊥).	`:- student(X), employee(X).`
IC5	Nobody is both a student and a visitor.	`:- student(X), visitor(X).`
IC6	Nobody is both an employee and a visitor.	`:- employee(X), visitor(X).`
IC7	Nobody is both faculty and staff.	`:- faculty(X), staff(X).`

5.4 Horn clause conversion

Each requirement went through four steps: an if-then sentence, an FOL formula, a rewrite so that the conclusion is a single atom, and Prolog syntax. Three examples:

Expired permits cannot authorize parking. Read positively: a permit is valid only if it is active, has started and has not expired. FOL: ∀p ∀b ∀e ∀d ((permit_status(p, active) ∧ permit_start(p, b) ∧ permit_expiry(p, e) ∧ current_date(d) ∧ b ≤ d ∧ e ≥ d) → permit_valid(p)). As a clause: ¬permit_status(p, active) ∨ ¬permit_start(p, b) ∨ ¬permit_expiry(p, e) ∨ ¬current_date(d) ∨ ¬(b ≤ d) ∨ ¬(e ≥ d) ∨ permit_valid(p), which has one positive literal and is therefore Horn (R06). An expired or not-yet-started permit never satisfies the body, so it can never authorize. The start date matters for Bilal: his one-day pass has the same start and expiry date, so it is valid on 2 October 2026 and on no other day.

A student may use a student bay. Taken literally this needs a disjunction over every kind of person and zone. Splitting by permit class and moving the pairing into the policy table grants/2 gives one Horn rule (R13) and three facts. A new zone type now needs one new fact, not a new rule.

Allocate the bay unless someone with higher priority wants it. "Unless" is negation, which a definite clause cannot contain. The positive part becomes loses(Y, S), derived when another eligible requester outranks Y (R25, R26). The decision R28 then uses negation as failure, `not loses(X, S)`. The rules are layered into strata: loses/2 and double_booked/1 are fully computed in stratum 0 before R28 runs in stratum 1, so "not" means "cannot be derived" and the result does not depend on rule order. This is the only place negation is used, and the stratifier rejects any rule set with recursion through negation.

Other negative requirements are handled positively. "Occupied bays cannot be allocated" becomes "only vacant bays, or bays reserved for the requester, can be allocated" (R19, R20), and IC3 confirms the outcome. The built-in comparisons ≥, >, < and ≠ are evaluated, never stored, as in Prolog.

5.5 Prolog form

The rule table in section 5.3 is printed from krr/knowledge_base/rules.py, the same objects the engines run, so it cannot differ from the code. The generated SWI-Prolog program prolog/smart_parking.pl holds the same 28 clauses in the same order. R01-R27 are written there exactly as in the table; the parts that change in Prolog are shown below: R28 writes `not` as `\+`, each denial constraint becomes a `violation/1` clause, and two helper predicates check consistency and print the allocations.

```
% H. Decision
% R28: Allocate the requested space if it can be allocated, the requester loses no conflict for it and is
      not double-booked.
allocate(X, S) :- requests(X, S, _T), can_allocate(X, S), \+ loses(X, S), \+ double_booked(X).

% ---- Integrity constraints (denial clauses) ----------------------
% A constraint is violated when its body can be proved.
% IC1: One space is never allocated to two people.
violation(ic1) :- allocate(X, S), allocate(Y, S), X \= Y.
% IC2: One person is never allocated two spaces.
violation(ic2) :- allocate(X, S1), allocate(X, S2), S1 \= S2.
% IC3: An occupied space is never allocated.
violation(ic3) :- allocate(_X, S), space_status(S, occupied).
% IC4: Nobody is both a student and an employee (Student ⊓ Employee ⊑ ⊥).
violation(ic4) :- student(X), employee(X).
% IC5: Nobody is both a student and a visitor.
violation(ic5) :- student(X), visitor(X).
% IC6: Nobody is both an employee and a visitor.
violation(ic6) :- employee(X), visitor(X).
% IC7: Nobody is both faculty and staff.
violation(ic7) :- faculty(X), staff(X).

% ---- Helpers -----------------------------------------------------
consistent :- \+ violation(_).
show_allocations :-
  forall(allocate(X, S), format("allocate(~w, ~w)~n", [X, S])).
```

6. INFERENCE AND REASONING

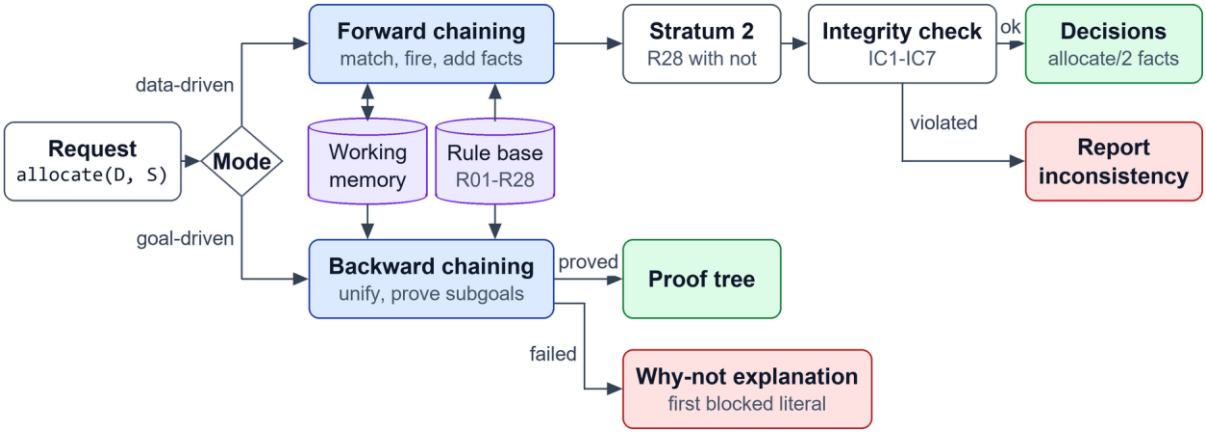


Figure 6: Inference engine architecture

Both engines read the same knowledge base. A request can be answered goal-first (backward) or data-first (forward). Forward results go through the integrity constraints before they are reported; backward results come back as a proof tree or, on failure, as a why-not explanation. None of the traces below is written by hand: each comes from running the engines.

6.1 Forward Chaining

6.1.1 Algorithm

Forward chaining is data-driven: it starts from the facts and fires rules until nothing new appears (krr/reasoning/forward.py).

```
procedure FORWARD-CHAIN(facts, rules)
    WM := facts
    for each stratum k = 0, 1 do                  # stratum 1 holds only R28
        repeat
            snapshot := copy(WM)
            conflict_set := { (r, θ) : r in rules of stratum k,
                              body(r)θ is true in snapshot }
            new := { head(r)θ : (r, θ) in conflict_set, head(r)θ not in WM }   # refraction
            log each firing (rule, bindings, premises, new fact)
            WM := WM ∪ new
        until new = ∅                             # fixpoint for this stratum
    check denial constraints IC1-IC7 on WM
    return WM
```

Because each cycle reads from a snapshot, a fact derived in cycle n is first used in cycle n + 1, so the cycle number is the length of the longest chain behind each fact. Termination is guaranteed: there are finitely many constants and no function symbols, so only finitely many ground atoms exist, and every productive cycle adds at least one.

6.1.2 Initial facts

The run starts from all 164 facts of section 5.2. The conclusion traced here, allocate(sara, s_a1), depends directly on these stored facts:

```
permit_status(p_sara, active)  permit_start(p_sara, 20260701)  permit_expiry(p_sara, 20270630)
current_date(20261002)  has_permit(sara, p_sara)  permit_class(p_sara, student_permit)
student(sara)  zone_opens(zone_a, 7)  zone_closes(zone_a, 22)
current_hour(10)  space_status(s_a1, vacant)  grants(student_permit, student_zone)
zone_type(zone_a, student_zone)  located_in(s_a1, zone_a)  owns(sara, car_sara)
vehicle_type(car_sara, car)  space_type(s_a1, standard)  fits(car, standard)
open_type(standard)  requests(sara, s_a1, 910)
```

6.1.3 Inference cycles

The main scenario reaches its fixpoint after 10 cycles:

Table 21: Forward chaining cycles

Cycle	Stratum	Conflict set	New facts	Rules fired
1	0	35	35	R01, R03, R04, R05, R06, R07, R09, R12, R17, R18, R23, R24
2	0	46	11	R02, R08, R10, R21, R22
3	0	56	10	R10, R11, R13
4	0	73	17	R11, R13, R14
5	0	87	14	R14, R15, R16
6	0	96	9	R15, R19, R20
7	0	104	6	R19, R20, R25
8	0	104	0	none (fixpoint)
9	1	3	3	R28
10	1	3	0	none (fixpoint)

Cycles 1-7 derive new facts and cycle 8 finds none, so stratum 0 is complete. Cycle 9 runs R28 in stratum 1 and cycle 10 confirms the fixpoint.

6.1.4 Rules activated and new facts

The full run has 105 firings (section 6.1.6). The trace sheet below keeps only the firings that the conclusion allocate(sara, s_a1) depends on, in the order they happened.

Table 22: Forward trace for allocate(sara, s_a1)

Step	Cycle / stratum	Facts before (premises)	Rule	New fact	Reason
11	1 / 0	permit_status(p_sara, active) ∧ permit_start(p_sara, 20260701) ∧ permit_expiry(p_sara, 20270630) ∧ current_date(20261002) ∧ 20260701 =< 20261002 ∧ 20270630 >= 20261002	R06	permit_valid(p_sara)	A permit is valid if its status is active and today lies between its start date and its expiry date.
16	1 / 0	has_permit(sara, p_sara) ∧ permit_class(p_sara, student_permit) ∧ student(sara)	R07	permit_matches_role(sara, p_sara)	A student permit only counts when its holder is a student.
21	1 / 0	zone_opens(zone_a, 7) ∧ zone_closes(zone_a, 22) ∧ current_hour(10) ∧ 10 >= 7 ∧ 10 < 22	R12	zone_open(zone_a)	A zone is open when the current hour is inside its opening hours.
25	1 / 0	space_status(s_a1, vacant)	R17	space_free(s_a1)	A vacant space is free.
42	2 / 0	permit_matches_role(sara, p_sara) ∧ permit_valid(p_sara)	R10	authorized(sara, p_sara)	A person is authorized by a permit that matches their role and is valid.
54	3 / 0	authorized(sara, p_sara) ∧ permit_class(p_sara, student_permit) ∧ grants(student_permit, student_zone) ∧ zone_type(zone_a, student_zone) ∧ zone_open(zone_a)	R13	zone_access(sara, zone_a)	An authorized person may enter a zone if their permit class grants that zone type and the zone is open.
62	4 / 0	zone_access(sara, zone_a) ∧ located_in(s_a1, zone_a) ∧ owns(sara, car_sara) ∧ vehicle_type(car_sara, car) ∧ space_type(s_a1, standard) ∧ fits(car, standard)	R14	suitable_space(sara, s_a1)	A space suits a person if it is in a zone they may enter and their vehicle fits the bay.
78	5 / 0	suitable_space(sara, s_a1) ∧ space_type(s_a1, standard) ∧ open_type(standard)	R15	eligible_for_space(sara, s_a1)	A suitable standard, EV or motorbike bay needs nothing more.
92	6 / 0	eligible_for_space(sara, s_a1) ∧ space_free(s_a1)	R19	can_allocate(sara, s_a1)	A free space can be allocated to any person eligible for it.
103	9 / 1	requests(sara, s_a1, 910) ∧ can_allocate(sara, s_a1) ∧ not loses(sara, s_a1) ∧ not double_booked(sara)	R28	allocate(sara, s_a1)	Allocate the requested space if it can be allocated, the requester loses no conflict for it and is not double-booked.

6.1.5 Final conclusion

The decision is reached in seven levels: permit_valid and permit_matches_role, then authorized, zone_access, suitable_space, eligible_for_space, can_allocate and finally allocate. Three other students also asked for s_a1. R23 gives the other students priority 1 in cycle 1 and R21 gives Sara priority 3 in cycle 2. In cycle 7, once can_allocate(sara, s_a1) is known, R25 fires three times: Sara beats Usman, Ali and Zara (section 6.1.6 lists these firings). Usman loses as well even though he could never get the bay, because his request is still there. In stratum 1, R28 allocates s_a1 only to Sara, the one requester with no loses fact. The conclusion is allocate(sara, s_a1). The decisions for all eight requests are in section 8.1.1.

6.1.6 Complete list of firings

Every firing of the main scenario in order: step number, cycle, rule and derived fact. Premises and bindings for each step are in docs/generated/forward_trace_full.txt.

```
 1 C1 R01 person(sara)
 2 C1 R01 person(usman)
 3 C1 R01 person(ali)
 4 C1 R01 person(zara)
 5 C1 R03 person(bilal)
 6 C1 R03 person(hina)
 7 C1 R04 employee(ahmed)
 8 C1 R05 employee(fatima)
 9 C1 R06 permit_valid(p_ahmed)
10 C1 R06 permit_valid(p_fatima)
11 C1 R06 permit_valid(p_sara)
12 C1 R06 permit_valid(vp_bilal)
13 C1 R06 permit_valid(p_ali)
14 C1 R06 permit_valid(p_zara)
15 C1 R07 permit_matches_role(zara, p_zara)
16 C1 R07 permit_matches_role(sara, p_sara)
17 C1 R07 permit_matches_role(ali, p_ali)
18 C1 R07 permit_matches_role(usman, p_usman)
54 C3 R13 zone_access(sara, zone_a)
55 C3 R13 zone_access(ali, zone_a)
56 C3 R13 zone_access(bilal, zone_v)
57 C4 R11 authorized_driver(fatima)
58 C4 R11 authorized_driver(ahmed)
59 C4 R13 zone_access(fatima, zone_e)
60 C4 R13 zone_access(ahmed, zone_e)
61 C4 R14 suitable_space(sara, s_a2)
62 C4 R14 suitable_space(sara, s_a1)
63 C4 R14 suitable_space(sara, s_a3)
64 C4 R14 suitable_space(sara, s_a5)
65 C4 R14 suitable_space(zara, s_a2)
66 C4 R14 suitable_space(zara, s_a1)
67 C4 R14 suitable_space(zara, s_a3)
68 C4 R14 suitable_space(zara, s_a5)
69 C4 R14 suitable_space(ali, s_a2)
70 C4 R14 suitable_space(ali, s_a1)
71 C4 R14 suitable_space(ali, s_a3)
```

```
19 C1 R09 permit_matches_role(bilal, vp_bilal)
20 C1 R12 zone_open(zone_v)
21 C1 R12 zone_open(zone_a)
22 C1 R12 zone_open(zone_e)
23 C1 R17 space_free(s_v1)
24 C1 R17 space_free(s_e1)
25 C1 R17 space_free(s_a1)
26 C1 R17 space_free(s_a3)
27 C1 R17 space_free(s_e2)
28 C1 R17 space_free(s_a4)
29 C1 R18 holds_reservation(zara, s_a5)
30 C1 R18 holds_reservation(ahmed, s_e3)
31 C1 R23 priority(usman, 1)
32 C1 R23 priority(ali, 1)
33 C1 R23 priority(zara, 1)
34 C1 R24 priority(bilal, 1)
35 C1 R24 priority(hina, 1)
36 C2 R02 person(ahmed)
37 C2 R02 person(fatima)
38 C2 R08 permit_matches_role(fatima, p_fatima)
39 C2 R08 permit_matches_role(ahmed, p_ahmed)
40 C2 R10 authorized(zara, p_zara)
41 C2 R10 authorized(ali, p_ali)
42 C2 R10 authorized(sara, p_sara)
43 C2 R10 authorized(bilal, vp_bilal)
44 C2 R21 priority(sara, 3)
45 C2 R22 priority(ahmed, 2)
46 C2 R22 priority(fatima, 2)
47 C3 R10 authorized(fatima, p_fatima)
48 C3 R10 authorized(ahmed, p_ahmed)
49 C3 R11 authorized_driver(zara)
50 C3 R11 authorized_driver(sara)
51 C3 R11 authorized_driver(ali)
52 C3 R11 authorized_driver(bilal)
53 C3 R13 zone_access(zara, zone_a)
72 C4 R14 suitable_space(ali, s_a5)
73 C4 R14 suitable_space(bilal, s_v1)
74 C5 R14 suitable_space(ahmed, s_e3)
75 C5 R14 suitable_space(ahmed, s_e2)
76 C5 R14 suitable_space(ahmed, s_e1)
77 C5 R15 eligible_for_space(bilal, s_v1)
78 C5 R15 eligible_for_space(sara, s_a1)
79 C5 R15 eligible_for_space(sara, s_a5)
80 C5 R15 eligible_for_space(ali, s_a5)
81 C5 R15 eligible_for_space(ali, s_a2)
82 C5 R15 eligible_for_space(zara, s_a1)
83 C5 R15 eligible_for_space(zara, s_a2)
84 C5 R15 eligible_for_space(zara, s_a5)
85 C5 R15 eligible_for_space(sara, s_a2)
86 C5 R15 eligible_for_space(ali, s_a1)
87 C5 R16 eligible_for_space(sara, s_a3)
88 C6 R15 eligible_for_space(ahmed, s_e2)
89 C6 R15 eligible_for_space(ahmed, s_e3)
90 C6 R15 eligible_for_space(ahmed, s_e1)
91 C6 R19 can_allocate(sara, s_a3)
92 C6 R19 can_allocate(sara, s_a1)
93 C6 R19 can_allocate(zara, s_a1)
94 C6 R19 can_allocate(ali, s_a1)
95 C6 R19 can_allocate(bilal, s_v1)
96 C6 R20 can_allocate(zara, s_a5)
97 C7 R19 can_allocate(ahmed, s_e1)
98 C7 R19 can_allocate(ahmed, s_e2)
99 C7 R20 can_allocate(ahmed, s_e3)
100 C7 R25 loses(ali, s_a1)
101 C7 R25 loses(zara, s_a1)
102 C7 R25 loses(usman, s_a1)
103 C9 R28 allocate(sara, s_a1)
104 C9 R28 allocate(bilal, s_v1)
105 C9 R28 allocate(ahmed, s_e3)
```

6.2 Backward Chaining

6.2.1 Algorithm

Backward chaining is goal-driven: it starts from a hypothesis and works back to the facts that support it. The engine is SLD resolution, the method Prolog uses (krr/reasoning/backward.py).

```
procedure SOLVE(goals, θ, ancestors)
    if goals is empty then yield θ; return
    G := first(goals) θ
    if G is a comparison: if it evaluates true, SOLVE(rest, θ)       # built-in
    else if G = not A:    if SOLVE([A], θ) has no answer, SOLVE(rest, θ)
    else if G is a variant of one of its ancestors: fail             # loop check
    else
        for each stored fact F that unifies with G (mgu σ): SOLVE(rest, θσ)
        for each rule H :- B with variables renamed apart:
            if H unifies with G (mgu σ): SOLVE(B + rest, θσ, ancestors + G)
        # trying the next fact or rule after a failure is backtracking
```

Subgoals are proved depth first, left to right. Ground queries stop at the first proof and queries with variables collect every answer. Each step is logged as GOAL, RULE, FACT, BUILTIN, NOT, PROVED or FAIL, with its depth.

6.2.2 Successful proof

Test case TC1: Ali asks for the vacant student bay s_a1 and nobody else wants it. The query is `?- allocate(ali, s_a1).` Each row is one rule application in the proof, with the supporting facts that verify it; the dots show depth. The full trace is in docs/generated/backward_ali_tc01.txt.

Table 23: Backward trace for allocate(ali, s_a1)

Goal	Rule	Subgoals	Evidence (facts, built-ins)	Result
allocate(ali, s_a1)	R28	requests(ali, s_a1, 900), can_allocate(ali, s_a1), not loses(ali, s_a1), not double_booked(ali)	requests(ali, s_a1, 900)	success
· can_allocate(ali, s_a1)	R19	eligible_for_space(ali, s_a1), space_free(s_a1)	(all derived)	success
· · eligible_for_space(ali, s_a1)	R15	suitable_space(ali, s_a1), space_type(s_a1, standard), open_type(standard)	space_type(s_a1, standard), open_type(standard)	success

Goal	Rule	Subgoals	Evidence (facts, built-ins)	Result
· · · suitable_space(ali, s_a1)	R14	zone_access(ali, zone_a), located_in(s_a1, zone_a), owns(ali, car_ali), vehicle_type(car_ali, car), space_type(s_a1, standard), fits(car, standard)	located_in(s_a1, zone_a), owns(ali, car_ali), vehicle_type(car_ali, car), space_type(s_a1, standard), fits(car, standard)	success
· · · · zone_access(ali, zone_a)	R13	authorized(ali, p_ali), permit_class(p_ali, student_permit), grants(student_permit, student_zone), zone_type(zone_a, student_zone), zone_open(zone_a)	permit_class(p_ali, student_permit), grants(student_permit, student_zone), zone_type(zone_a, student_zone)	success
· · · · · authorized(ali, p_ali)	R10	permit_matches_role(ali, p_ali), permit_valid(p_ali)	(all derived)	success
· · · · · · permit_matches_role(ali, p_ali)	R07	has_permit(ali, p_ali), permit_class(p_ali, student_permit), student(ali)	has_permit(ali, p_ali), permit_class(p_ali, student_permit), student(ali)	success
· · · · · · permit_valid(p_ali)	R06	permit_status(p_ali, active), permit_start(p_ali, 20260701), permit_expiry(p_ali, 20270630), current_date(20261002), 20260701 =< 20261002, 20270630 >= 20261002	permit_status(p_ali, active), permit_start(p_ali, 20260701), permit_expiry(p_ali, 20270630), current_date(20261002), 20260701 =< 20261002, 20270630 >= 20261002	success
· · · · · zone_open(zone_a)	R12	zone_opens(zone_a, 7), zone_closes(zone_a, 22), current_hour(10), 10 >= 7, 10 < 22	zone_opens(zone_a, 7), zone_closes(zone_a, 22), current_hour(10), 10 >= 7, 10 < 22	success
· · space_free(s_a1)	R17	space_status(s_a1, vacant)	space_status(s_a1, vacant)	success
· not loses(ali, s_a1)	NAF	try to prove loses(ali, s_a1)	no proof found	success
· not double_booked(ali)	NAF	try to prove double_booked(ali)	no proof found	success

The leaves of the proof are 21 stored facts and 3 comparisons. The two "not" goals succeed because no competing request and no second reservation exist, so loses(ali, s_a1) and double_booked(ali) cannot be proved.

6.2.3 Failed proof with backtracking

A failed proof shows backtracking. For `?- authorized_driver(usman).` R11 reduces the goal to authorized(usman, P) and R10 to permit_matches_role and permit_valid. R07 succeeds with P = p_usman, but permit_valid(p_usman) fails at the comparison 20260915 >= 20261002. The engine backtracks and tries R08 and R09 for permit_matches_role; both fail because p_usman is neither an employee permit nor a visitor pass. With no alternatives left, the goal fails, and the explainer reports "permit_valid(p_usman) fails at 20260915 >= 20261002". The full trace:

```
?- authorized_driver(usman).

  1  GOAL  authorized_driver(usman)
  2  RULE  try R11: authorized_driver(X) :- authorized(X, P).
  3     subgoals: authorized(usman, P_1)
  4   GOAL  authorized(usman, P_1)
  5   RULE  try R10: authorized(X, P) :- permit_matches_role(X, P), permit_valid(P).
  6       subgoals: permit_matches_role(usman, P_2), permit_valid(P_2)
  7     GOAL  permit_matches_role(usman, P_2)
  8     RULE  try R07: permit_matches_role(X, P) :- has_permit(X, P), permit_class(P, student_permit),
        student(X).
  9         subgoals: has_permit(usman, P_3), permit_class(P_3, student_permit), student(usman)
 10       GOAL  has_permit(usman, P_3)
 11       FACT  has_permit(usman, p_usman)  matches the stored fact
 12       GOAL  permit_class(p_usman, student_permit)
 13       FACT  permit_class(p_usman, student_permit)  matches the stored fact
 14       GOAL  student(usman)
 15       FACT  student(usman)  matches the stored fact
 16     PROVED  permit_matches_role(usman, p_usman)  by R07
 17     GOAL  permit_valid(p_usman)
 18     RULE  try R06: permit_valid(P) :- permit_status(P, active), permit_start(P, B), permit_expiry(P,
       E), current_date(D), B =< D, E >= D.
 19         subgoals: permit_status(p_usman, active), permit_start(p_usman, B_4), permit_expiry(p_usman,
       E_4), current_date(D_4), B_4 =< D_4, E_4 >= D_4
 20       GOAL  permit_status(p_usman, active)
 21       FACT  permit_status(p_usman, active)  matches the stored fact
 22       GOAL  permit_start(p_usman, B_4)
 23       FACT  permit_start(p_usman, 20250916)  matches the stored fact
 24       GOAL  permit_expiry(p_usman, E_4)
 25       FACT  permit_expiry(p_usman, 20260915)  matches the stored fact
 26       GOAL  current_date(D_4)
 27       FACT  current_date(20261002)  matches the stored fact
 28       BUILTIN 20250916 =< 20261002  is true
```

```
  29       BUILTIN 20260915 >= 20261002  is false
  30     FAIL  permit_valid(p_usman)  (no fact matches and no rule succeeds)
  31     RULE  try R08: permit_matches_role(X, P) :- has_permit(X, P), permit_class(P, employee_permit),
       employee(X).
  32         subgoals: has_permit(usman, P_5), permit_class(P_5, employee_permit), employee(usman)
  33       GOAL  has_permit(usman, P_5)
  34       FACT  has_permit(usman, p_usman)  matches the stored fact
  35       GOAL  permit_class(p_usman, employee_permit)
  36       FAIL  permit_class(p_usman, employee_permit)  (no fact matches and no rule succeeds)
  37     RULE  try R09: permit_matches_role(X, P) :- has_permit(X, P), permit_class(P, visitor_pass),
       visitor(X).
  38         subgoals: has_permit(usman, P_6), permit_class(P_6, visitor_pass), visitor(usman)
  39       GOAL  has_permit(usman, P_6)
  40       FACT  has_permit(usman, p_usman)  matches the stored fact
  41       GOAL  permit_class(p_usman, visitor_pass)
  42       FAIL  permit_class(p_usman, visitor_pass)  (no fact matches and no rule succeeds)
  43   FAIL  authorized(usman, P_1)  (no fact matches and no rule succeeds)
  44  FAIL  authorized_driver(usman)  (no fact matches and no rule succeeds)

authorized_driver(usman): FAILED. No proof exists from the facts and rules.

authorized_driver(usman) cannot be proved. Reasons, rule by rule:
  R11 blocked at authorized(usman, P)
  R10 blocked at permit_valid(p_usman)
   R06 blocked at 20260915 >= 20261002  (false)
```

Queries with variables return every answer: `?- can_allocate(ahmed, S).` gives S = s_e1, s_e2 and s_e3 (s_e2 is an EV bay that fits his electric car and s_e3 is reserved for him today).

7. IMPLEMENTATION

7.1 Tools and Technologies

The system is written in Python 3.10 or newer (tested on 3.10, 3.12 and 3.14). The reasoning engines, CLI and tests use only the standard library, so anyone can run them without installing anything and every line of the reasoning is ours to explain in the viva. Streamlit provides the web interface, python-docx builds this report, Pillow draws its figures at print size (tools/figures.py), Mermaid draws the diagrams kept in the repository, and the knowledge base is also exported to SWI-Prolog and to OWL 2 (Turtle) for Protégé. The project website uses Three.js for its 3D scene and Pyodide to run the engine in the browser (section 7.2.9). GitHub Actions runs the tests on every push.

7.2 Code / Execution

7.2.1 Project structure

```
smart-parking-krr/   LICENSE, README.md, app.py, main.py, requirements.txt
  docs/        diagrams/, figures/, generated/, report/, screenshots/, viva/
  krr/         site_export.py, system.py
  core/       kb.py, terms.py
  export/      diagrams.py, owl.py, prolog.py
  frames/      frames.py, ontology.py, scripts.py
  knowledge_base/  facts.py, rules.py, scenarios.py
  logic/       description_logic.py, fol.py
  reasoning/     backward.py, explain.py, forward.py
  semantic_net/   network.py
  ontology/      smart_parking.ttl
  prolog/       smart_parking.pl
  tests/        test_consistency.py, test_core.py, test_prolog.py, test_reasoning.py,
       test_representation.py, test_site_export.py
  tools/        build_all.py, build_report.py, build_site.py, figures.py, finalize_report.py
  website/       app.js, data.js, engine-worker.js, engine.zip, index.html, scene3d.js, styles.css
  assets/      architecture.png, frame_hierarchy.png, parking_script.png, reasoning_engine.png,
       semantic_network.png, semantic_network_example.png
  models/      LICENSE-kenney-car-kit.txt, cone.glb, hatchback-sports.glb, sedan-sports.glb, sedan.glb,
       suv-luxury.glb, suv.glb, van.glb
   Textures/    colormap.png
```

7.2.2 Knowledge base implementation

An atom is a Python tuple such as ("owns", "ali", "car_ali"). Tuples are hashable, so facts are kept in sets indexed by predicate name. A variable is a string that starts with an upper-case letter, and a substitution is a dictionary from variables to terms. Rules are written as Prolog text in krr/knowledge_base/rules.py, parsed into Rule objects and checked for safety when the module loads. Unification is the standard algorithm for function-free terms (krr/core/terms.py):

```
def unify_terms(a, b, subst):
    a, b = resolve(a, subst), resolve(b, subst)
    if a == b:
        return subst
    if is_var(a):
        return {**subst, a: b}
    if is_var(b):
        return {**subst, b: a}
    return None              # two different constants never unify
```

7.2.3 Frame implementation

Slot, Frame and FrameSystem are dataclasses in krr/frames/frames.py. FrameSystem.get() implements the lookup order of section 3.1.1 and returns the value together with its source (own value, default from a named frame, or if-needed). If-needed procedures are ordinary Python functions stored in the slot. FrameSystem.to_facts() turns instance frames into knowledge base facts using the predicate facet.

7.2.4 Forward chaining implementation

One cycle takes a snapshot of working memory, matches every rule body against it and keeps only new heads:

```
snapshot = {p: set(work.facts_for(p)) for p in work.predicates}
for rule in rules:
    for subst in match_body(rule.body, work, {}, snapshot):
        conflict_set += 1
        head = substitute(rule.head, subst)
        if head in work or head in new_this_cycle:
            continue                       # refraction: nothing new
        new_this_cycle[head] = Firing(step, cycle_no, stratum, rule, subst, premises, head)
```

The stratifier assigns R28 to stratum 1 because its body has "not", and raises an error if a rule set contains recursion through negation.

7.2.5 Backward chaining implementation

The solver is a Python generator, so backtracking is just asking it for the next answer:

```
for rule in self.kb.rules_for(goal[0]):
    renamed = rule.renamed(str(next(self._counter)))   # standardise apart
    s2 = unify(goal, renamed.head, subst)
    if s2 is None:
        continue
    for s3, children in self._solve_body(renamed.body, s2, depth + 1, ancestors + (goal,)):
        yield s3, ProofNode(format_atom(substitute(goal, s3)), rule.rid, children)
```

Renaming is why traces show variables such as P_5. A depth limit of 40 is a second guard after the loop check.

7.2.6 User interaction and sample queries

```
python main.py demo          python main.py prove "can_allocate(ahmed, S)"    python main.py cases
python main.py forward       python main.py why "allocate(usman, s_a1)"       streamlit run app.py
```

The Streamlit interface has ten tabs: overview, knowledge base, frames and script, semantic network, FOL, DL, forward chaining (with a slider that replays inference cycle by cycle), backward chaining (free-text queries with trace, drawn proof tree and why-not), test cases, and a playground where the user changes the date, hour and requests and runs the reasoner again.

7.2.7 Sample execution

Output of `python main.py why "allocate(fatima, s_e1)" -s tc06_no_suitable_space`:

```
allocate(fatima, s_e1) cannot be proved. Reasons, rule by rule:
  R28 blocked at can_allocate(fatima, s_e1)
    R19 blocked at eligible_for_space(fatima, s_e1)
      R15 blocked at suitable_space(fatima, s_e1)
        R14 blocked at fits(motorbike, standard)  (no such fact is stored)
      R16 blocked at suitable_space(fatima, s_e1)  (explained above)
    R20 blocked at eligible_for_space(fatima, s_e1)  (explained above)
```

The output of `python main.py cases` is summarised in section 8.1.2.

7.2.8 Execution screenshots

Screenshots of the Streamlit interface running the main scenario, followed by two from the project website.

Decisions for this scenario

Driver	Space	Decision	Reason (from backward chaining)
ahmed	s_e3	Allocated	
fatima	s_e1	Denied	suitable_space(fatima, s_e1) fails at fits(motorbike, standard) (no such fact is stored)
zara	s_a1	Denied	allocate(zara, s_a1) fails at not loses(zara, s_a1) (loses(zara, s_a1) is provable by R25; compe
usman	s_a1	Denied	permit_valid(p_usman) fails at 20260915 >= 20261002 (false)
ali	s_a1	Denied	allocate(ali, s_a1) fails at not loses(ali, s_a1) (loses(ali, s_a1) is provable by R25; competing r
sara	s_a1	Allocated	
hina	s_v1	Denied	permit_matches_role(hina, P) fails at has_permit(hina, P) (no such fact is stored)
bilal	s_v1	Allocated	

Figure 7: Overview tab: decisions for the morning rush

Forward chaining

Cycle	Stratum	Conflict set	New facts	Rules fired
1	0	35	35	R01, R03, R04, R05, R06, R07, R09, R12,
2	0	46	11	R02, R08, R10, R21, R22
3	0	56	10	R10, R11, R13
4	0	73	17	R11, R13, R14
5	0	87	14	R14, R15, R16
6	0	96	9	R15, R19, R20
7	0	104	6	R19, R20, R25
8	0	104	0	
9	1	3	3	R28
10	1	3	0	

Figure 8: Forward chaining tab: cycle table

Backward chaining on `allocate(ali, s_a1)`

```
allocate(ali, s_a1)   [R28]
├── requests(ali, s_a1, 900)   [fact]
├── can_allocate(ali, s_a1)   [R19]
│   ├── eligible_for_space(ali, s_a1)   [R15]
│   │   ├── suitable_space(ali, s_a1)   [R14]
│   │   │   ├── zone_access(ali, zone_a)   [R13]
│   │   │   │   ├── authorized(ali, p_ali)   [R10]
│   │   │   │   │   ├── permit_matches_role(ali, p_ali)   [R07]
│   │   │   │   │   │   ├── has_permit(ali, p_ali)   [fact]
│   │   │   │   │   │   ├── permit_class(p_ali, student_permit)   [fact]
│   │   │   │   │   │   └── student(ali)   [fact]
│   │   │   │   │   └── permit_valid(p_ali)   [R06]
│   │   │   │   │       ├── permit_status(p_ali, active)   [fact]
│   │   │   │   │       ├── permit_expiry(p_ali, 20270630)   [fact]
│   │   │   │   │       ├── current_date(20261002)   [fact]
│   │   │   │   │       └── 20270630 >= 20261002   [built-in]
```

Figure 9: Test cases tab: proof tree for allocate(ali, s_a1)

Test cases

Case	Checks	Forward = backward = expected	Constraint violations
TC1 Valid student, valid permit, free student bay	4	yes	
TC2 Expired permit	3	yes	
TC3 Visitor without authorization	3	yes	
TC4 Space already occupied	4	yes	
TC5 Student requests an employee-only space	3	yes	
TC6 Eligible driver, but no suitable space	4	yes	
TC7 Three drivers request the same space	5	yes	
TC8 Accessible space requested by an ineligible driver	4	yes	
TC9 Conflicting reservations	3	yes	
TC10 Visitor zone closed at 19:00	3	yes	

Figure 10: Test cases tab: both engines on all cases

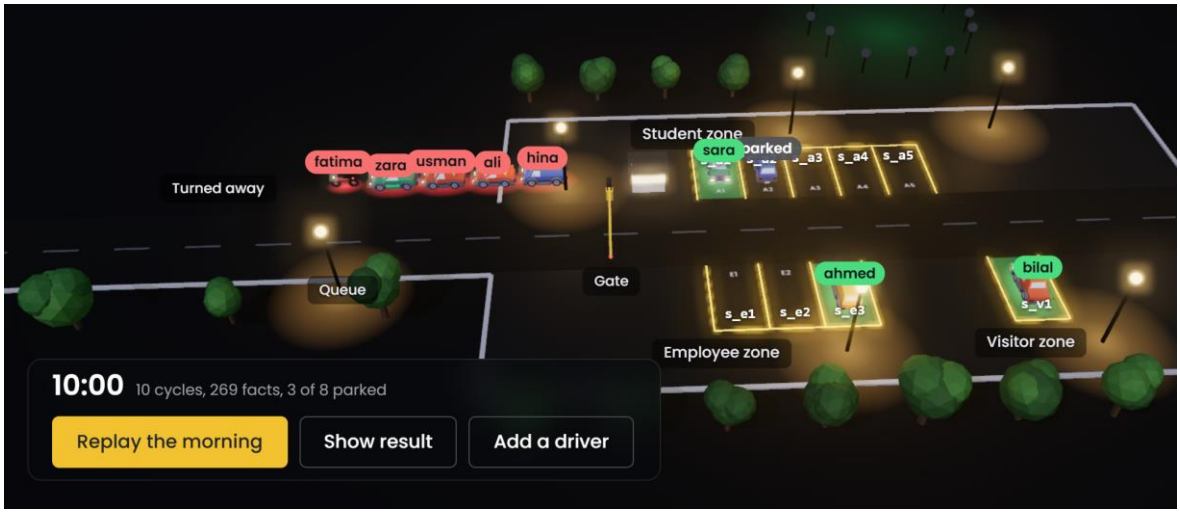


Figure 11: Project website: the 3D car park after the morning rush (intro panel hidden)

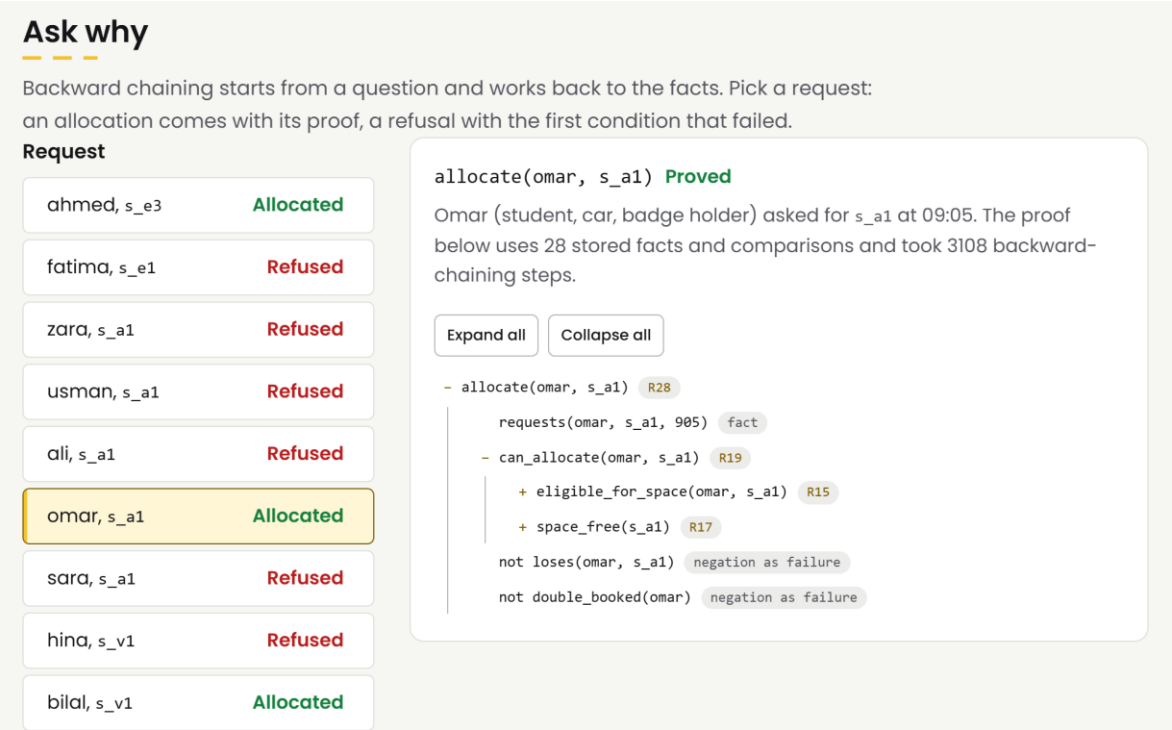


Figure 12: Project website: Ask why for omar, a driver added through the form

7.2.9 Project website

The project also has a public website at https://smart-parking-krr.vercel.app. The page itself is static: tools/build_site.py runs the engines and writes every decision, proof, inference cycle and test result into a data file, so what the site shows is the code's own output. The car park is drawn in 3D with Three.js, and a flat 2D version is used when the browser has no WebGL or the visitor has asked for reduced motion.

The 3D view replays the morning rush. The eight cars arrive, the forward chainer runs its 10 cycles, and each car is checked at the gate. Allocated cars reverse into their bay, and refused cars turn away with the root cause found by the explainer. Further down the page the camera visits the knowledge core (the forward-chaining cycles), a proof tower for the backward-chaining answer that is selected, the semantic network and the twelve test cases.

Two features run the real engine in the browser. Pyodide, a build of CPython for WebAssembly, loads the krr package from a zip file next to the page, so no server is involved.

- Scenario clock. Any date and hour can be chosen. krr/site_export.py rebuilds the main scenario with new current_date and current_hour facts and runs forward chaining again. At 23:00, for example, every zone is closed and no request is allocated, and on 3 October 2026 Bilal's one-day pass fails R06.
- Add a driver. A form asks for name, role, vehicle, permit state, bay, arrival time and accessibility badge. The answers become the same kind of facts the frames give the built-in drivers, such as student(omar), owns(omar, veh_omar), permit_start(pm_omar, D) and requests(omar, s_a1, 905). The engine then decides every request again, and Ask why shows the new driver's proof or the first condition that failed (section 7.2.8).

The second feature lets an examiner describe a new case and see it answered with a proof built from our rules. tests/test_site_export.py checks both features with the same unittest suite as the rest of the system.

8. RESULTS AND DISCUSSION

8.1 Results

All numbers below are produced by running the code when this report is built.

8.1.1 Decisions for the main scenario

Forward chaining on the morning rush gives these decisions, with reasons from the explainer:

Table 24: Decisions for the morning rush

Driver	Space	Decision	Reason from the explainer
ahmed	s_e3	allocated	-
fatima	s_e1	denied	suitable_space(fatima, s_e1) fails at fits(motorbike, standard) (no such fact is stored)
zara	s_a1	denied	allocate(zara, s_a1) fails at not loses(zara, s_a1) (loses(zara, s_a1) is provable by R25; competing request: requests(sara, s_a1, 910))
usman	s_a1	denied	permit_valid(p_usman) fails at 20260915 >= 20261002 (false)
ali	s_a1	denied	allocate(ali, s_a1) fails at not loses(ali, s_a1) (loses(ali, s_a1) is provable by R25; competing request: requests(sara, s_a1, 910))
sara	s_a1	allocated	-
hina	s_v1	denied	permit_matches_role(hina, P) fails at has_permit(hina, P) (no such fact is stored)
bilal	s_v1	allocated	-

8.1.2 Test cases

Testing works at four levels. Unit tests cover the parser, unification, rule safety, frames, the semantic network, the DL reasoner and the FOL model checker. Rule tests check single rules on small fact sets. Inference tests check that the forward closure is a model of every rule and that every firing uses only facts known before it. Scenario tests run twelve cases and compare each expected goal with both engines.

Table 25: Test cases run through both engines

TC	Scenario	Input facts	Expected	Actual	Status
TC-01	Valid student, valid permit, free student bay	student(ali), permit p_ali valid to 20270630, s_a1 vacant	allocate(ali, s_a1) is true	forward true, backward true (4 goals checked)	PASS
TC-02	Expired permit	permit_expiry(p_usman, 20260915), today 20261002	authorized_driver(usman) is false	forward false, backward false (3 goals checked)	PASS
TC-03	Visitor without authorization	visitor(hina), no has_permit fact	authorized_driver(hina) is false	forward false, backward false (3 goals checked)	PASS
TC-04	Space already occupied	requests(ali, s_a2), space_status(s_a2, occupied)	can_allocate(ali, s_a2) is false	forward false, backward false (4 goals checked)	PASS
TC-05	Student requests an employee-only space	requests(ali, s_e1), zone_e is employee_zone	allocate(ali, s_e1) is false	forward false, backward false (3 goals checked)	PASS
TC-06	Eligible driver, but no suitable space	fatima owns motorbike, s_e1 is standard	allocate(fatima, s_e1) is false	forward false, backward false (4 goals checked)	PASS
TC-07	Three drivers request the same space	zara 08:50, ali 09:05, sara 09:10 (badge) all request s_a1	allocate(ali, s_a1) is false	forward false, backward false (5 goals checked)	PASS
TC-08	Accessible space requested by an ineligible driver	requests(ali, s_a3), accessibility_badge(ali, no)	eligible_for_space(ali, s_a3) is false	forward false, backward false (4 goals checked)	PASS
TC-09	Conflicting reservations	reserved_by(r2, ahmed), reserved_by(r3, ahmed), same date	allocate(ahmed, s_e3) is false	forward false, backward false (3 goals checked)	PASS
TC-10	Visitor zone closed at 19:00	current_hour(19), zone_v open 8-17	allocate(bilal, s_v1) is false	forward false, backward false (3 goals checked)	PASS
TC-11	Reserved space requested by someone else	s_a5 reserved by zara (r1), ali requests it	can_allocate(ali, s_a5) is false	forward false, backward false (3 goals checked)	PASS
TC-12	Inconsistent data: a student recorded as faculty	student(ali) plus faculty(ali)	employee(ali) is true	forward true, backward true (2 goals checked)	PASS

Every case passes with both engines. The cases exercise each requirement of section 2.1.3 at least once, and each denial is caught at the right rule: an expired permit at R06 (TC2), a missing pass at R09 (TC3), an occupied bay at R19/R20 (TC4), the wrong zone at R13 (TC5), a vehicle that does not fit at R14 (TC6), the missing badge at R16 (TC8), a closed zone at R12 (TC10) and someone else's reservation at R20 (TC11). TC7 checks both conflict rules, priority (R25) and arrival time (R26). TC9 checks the only use of negation. TC12 checks that bad data is caught, independently, by IC4 and by DL axiom D01.

8.1.3 Inference accuracy and consistency

- All 12 test cases and the main scenario give the expected result with both engines: 49 goal checks, each answered the same way by forward and backward chaining.

- For every derived predicate in all 13 scenarios, backward chaining with all variables free returns exactly the set of facts forward chaining derives. The engines are written independently, so this is the strongest check we have.
- All 12 FOL statements hold on the main model; F04 fails on the TC12 data with the counterexample x = ali, as it should.
- The DL ABox of the main scenario is consistent, and DL realization of AuthorizedDriver matches authorized_driver for all eight people.
- The generated Prolog program is parsed back by a unit test and contains exactly the same facts and rules. Loaded in SWI-Prolog 10, it proves exactly the conclusions the Python forward chainer derives in all 13 scenarios, including the IC4 violation in TC12, and it loads without warnings. `python main.py prolog` repeats this check, and GitHub Actions runs it on every push.
- The suite has 70 unit tests, all passing.

We did not measure speed or accuracy on real traffic, because there is no real traffic data; the claims above are about logical correctness on our scenarios.

8.2 Discussion

8.2.1 Whether the objectives were met

The system meets the five objectives of section 1.2. It has 15 class frames, a running script and a semantic network generated from the frames (objective 1). All 28 rules have an FOL form and twelve further statements are checked on the data (objective 2). The TBox has 45 axioms, and the reasoner classifies them and reports contradictions (objective 3). Both chaining engines run the 28 Horn rules and write step-by-step traces (objective 4). Twelve test cases pass with both engines, and the engines derive the same facts in every scenario (objective 5).

8.2.2 Expressivity against tractability

Propositional logic has no objects or variables. "A student with a valid student permit may enter a student zone" must be written once for every student, permit and zone: ali_valid ∧ ali_student → ali_zone_a, and so on. With 8 people, 7 permits and 3 zones, R13 alone becomes dozens of rules, and each new student needs new ones. It also cannot say "every bay is in some zone" (F01) or compare an expiry date with today. Inference is decidable, and propositional Horn clauses can even be decided in linear time, but the knowledge base would grow with every person and bay.

In FOL, quantified variables let one rule cover every individual, so the 28 rules work for any number of drivers and bays. FOL can also state F01 (existential), F02 (equality), F03 (disjunction) and F10 (negation inside a quantifier), none of which propositional logic can say in general form.

The cost is that entailment in full FOL is only semi-decidable: a prover may run forever on a formula that does not follow. Even when it stops, resolution in full FOL has no useful time bound. A gate cannot wait for an answer that may never come. Our model checker avoids the problem because it evaluates the statements on one finite model, which always terminates; it does not prove that they follow from the rules in every model.

DL is a decidable fragment of FOL. Each DL axiom translates into FOL with at most two variables: ParkingSpace ⊑ ∃locatedIn.ParkingZone becomes ∀x (ParkingSpace(x) → ∃y (locatedIn(x, y) ∧ ParkingZone(y))). Satisfiability in ALC is PSPACE-complete, and in the EL fragment (only ⊓ and ∃), which covers most of our TBox, subsumption takes polynomial time (Baader, Brandt and Lutz, 2005). The price is that DL cannot join several variables in a chain. R25 compares the priorities of two requesters for the same bay, which needs three variables bound together, and the comparison PX > PY is not a DL constructor.

A Horn clause has at most one positive literal, so disjunctive and existential conclusions (F01, F03) cannot be written. In return the system gets what the gate needs. Without function symbols (Datalog) the set of derivable facts is finite, so forward chaining always terminates; ours stops after 10 cycles. Forward chaining over ground Horn clauses runs in polynomial time in the number of facts. Every answer has a proof made of rule applications, which is what the explainer shows. And Horn clauses run directly in Prolog. Stratified negation as failure adds "unless" without losing termination, at the cost of non-monotonicity: a new higher-priority request can remove an allocation.

8.2.3 How well the KR approaches work together

No single formalism does everything here. Frames handle defaults and overriding, which logic cannot do without non-monotonic extensions. The script captures the order of events. The semantic network makes the model readable and answers path questions. FOL states what must always be true, including statements that cannot be executed efficiently. DL keeps the terminology consistent and supports the open-world reading. Horn clauses execute the policy and explain it. One vocabulary, checked by a unit test, lets them work as one system.

8.2.4 Forward against backward chaining

Forward chaining computes everything at once, which suits the morning batch: 105 firings decide all eight requests, though some of that work (such as Hina's priority) answers no question. Backward chaining touches only what the question needs and gives a proof tree directly, which suits one driver at the gate, but it can repeat work: proving allocate(sara, s_a1) takes 2,240 trace steps because the "not loses" subgoal re-proves can_allocate for every competitor. Tabling would remove that repetition. We use both, and their agreement is itself a test (section 8.1.3).

8.2.5 Limitations observed

- Time is coarse: whole hours and whole-day reservations. A zone closing at 17:30 cannot be expressed.
- Request times are assumed unique. Two equal-priority requests at the same minute would both be allocated; IC1 would report it, but no rule resolves it.
- One request per driver is assumed; IC2 would report a driver allocated two bays.
- Each run is a snapshot. Allocating a bay does not change its status inside the same run, because classical logic cannot retract facts; the script updates the status afterwards.
- The DL reasoner is structural: complete for the EL part of the TBox but incomplete for ⊔ and ∀. Full ALCI reasoning needs a tableau reasoner such as HermiT, which can be run on the exported OWL file in Protégé.
- The backward chainer has no tabling, so shared subgoals are recomputed.

9. CONCLUSION

9.1 Conclusion and Future Improvements

We modelled campus parking access and allocation in five paradigms that share one vocabulary: frames, a script, a semantic network, FOL and a DL ontology. Decisions come from 28 Horn rules with one stratified negation, run by a forward chainer that writes complete trace sheets and a backward chainer that proves single goals and explains every denial.

The system shows that one parking policy can be represented several ways and kept consistent, and that two independent engines reach the same decisions on every test. It explains every refusal down to the failing fact or comparison. What remains outside scope is real-time data, finer time, uncertainty and deployment at the gate.

Possible improvements:

1. Represent time intervals (Allen's interval relations or the event calculus) for hourly reservations and exact closing times.
2. After a loss, suggest the best free alternative bay; the query can_allocate(X, S) already lists the candidates.
3. Add tabling to the backward chainer.
4. Add certainty factors for sensor-reported occupancy, which is not always reliable.
5. Connect to live occupancy sensors and the permit database, keeping the rule base unchanged.
6. Add a truth maintenance mode so one new request does not recompute the whole closure.

10. REFERENCES

1. Brachman, R. J. and Levesque, H. J. (2004). Knowledge Representation and Reasoning. Morgan Kaufmann.
2. Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach, 4th edition. Pearson.

3. Baader, F., Calvanese, D., McGuinness, D. L., Nardi, D. and Patel-Schneider, P. F. (eds.) (2003). The Description Logic Handbook. Cambridge University Press.
4. Baader, F., Brandt, S. and Lutz, C. (2005). Pushing the EL envelope. Proceedings of IJCAI 2005, pp. 364-369.
5. Minsky, M. (1974). A Framework for Representing Knowledge. MIT AI Laboratory Memo 306.
6. Schank, R. C. and Abelson, R. P. (1977). Scripts, Plans, Goals and Understanding. Lawrence Erlbaum Associates.
7. Quillian, M. R. (1968). Semantic memory. In M. Minsky (ed.), Semantic Information Processing. MIT Press.
8. Kowalski, R. (1974). Predicate logic as programming language. Proceedings of IFIP Congress 74, pp. 569-574.
9. Clark, K. L. (1978). Negation as failure. In H. Gallaire and J. Minker (eds.), Logic and Data Bases. Plenum Press.
10. Apt, K. R., Blair, H. A. and Walker, A. (1988). Towards a theory of declarative knowledge. In J. Minker (ed.), Foundations of Deductive Databases and Logic Programming. Morgan Kaufmann.
11. W3C OWL Working Group (2012). OWL 2 Web Ontology Language Document Overview (Second Edition). https://www.w3.org/TR/owl2-overview/
12. SWI-Prolog Reference Manual. https://www.swi-prolog.org/pldoc/
13. Department of Computer Science and IT, NED University. CT-351 Knowledge Representation & Reasoning, Complex Computing Problem guide.

11. TEAM PARTICIPATION

Each member owned one part of the system from design to code to report section. All four of us reviewed the full rule base together before it was frozen, and every rule was agreed by the whole group.

Table 26: Team contributions

Student Name	Roll Number	Contribution / Responsibilities
Syed Mohmmad Huzaifa	AI-23022	Wrote the problem statement and the 13 domain requirements, ran the knowledge acquisition sessions, designed the 15 class frames and 38 instance frames with their facets and defaults, and wrote the ParkingSession script. Sections 1, 2, 3.1; krr/frames/.
Maaz ur Rehman	AI-23036	Built the semantic network generator with inheritance and path queries, wrote the DL TBox (T01-T28, D01-D17), the classifier and consistency checker, the OWL export and the expressivity and tractability analysis. Sections 3, 3.2, 4.2, 8.2; krr/semantic_net/, krr/logic/description_logic.py.
Owais Tariq	AI-23037	Wrote the FOL vocabulary, the rule readings and statements F01-F12 with the model checker, converted the policy into Horn rules R01-R28 and IC1-IC7, and built the forward chainer with stratified negation and the Prolog export. Sections 4.1, 5, 6.1; krr/logic/fol.py, krr/reasoning/forward.py.
Shaikh Muhammad Zain	AI-23306	Built the backward chainer, proof trees and why-not explainer, the CLI and Streamlit interface, the 12 test scenarios and the unit tests, and put the report together. Sections 6.2, 7, 8.1; krr/reasoning/backward.py, app.py, tests/.

In the viva each member presents and demonstrates their own part: Syed Mohmmad Huzaifa the problem, frames and script; Maaz ur Rehman the semantic network and DL reasoner; Owais Tariq the Horn rules and forward chaining; Shaikh Muhammad Zain backward chaining, the interface and the test cases. Each member also prepares the complete system for the individual viva.

Declaration: We confirm that the submitted report represents our group's work and that each member has contributed as stated above.

Group Representative: Shaikh Muhammad Zain (AI-23306) Date: 8 November 2026